
Faculty of Media Engineering and Technology
German University in Cairo

# ’GO-DRiVeS Mobile App for Passenger-Vehicle Real-Time Interaction with In-Campus Golf Car

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering

By

**Manuella Ehab Youssef Eskandar**

Supervised by

**Dr. Catherine Elias**

May 27, 2025

This is to certify that:

(i) the thesis comprises only my original work toward the Bachelor Degree of Science (B.Sc.) at the German University in Cairo (GUC),

(ii) due acknowledgment has been made in the text to all other material used

Manuella Ehab Youssef Eskandar
May 27, 2025

# Acknowledgments

I would like to thank the following for their efforts with me in my study and for supporting me during this journey.To begin, my parents and brother for their endless support at home and always believing in me. To Dr.Catherine, whose expertise and guidance was crucial and very beneficial for this project. To my teammates, for creating a new, fun exciting , environment where we got to work and help each other. Finally to my friends for their constant emotional support and encouragement.

# List of Publications

The following publications have been published while in conduct of this study:

- Authors. paper title. In *conference/journal* Status(under review, accepted to be published,Volume or pages in case of published), Year.

# Abstract

The transportation industry is undergoing a significant transformation with the rise of Autonomous Vehicles (AVs) and smart mobility solutions. This thesis presents the design and implementation of a ride-hailing application created specifically for AV deployment within a university campus setting. Inspired by the evolution of mobility, from early automobiles to today's autonomous systems, this work explores the intersection of AV technology and modern ride-hailing platforms.

A complete mobile application was developed, complete with driver and administrator roles. Live sensor data integration and manual driving were used in real-time simulations to test the system. Key features such as GPS-based localization, camera integration, real-time notifications, and admin control panels were implemented and evaluated.

Through a series of organized development sprints, the application achieved an 80% success rate across simulated test rides. User feedback highlighted the app's usability, with suggestions for further refinement. The results confirm the feasibility of a modular, campus-specific ride-hailing app for AV systems and lay the foundation for full autonomy and broader deployment in future work.

# Contents

# List of Abbreviations

| | |
|---|---|
| **EVs** | Electric Vehicles |
| **AV** | Autonomous Vehicle |
| **MaaS** | Mobility as a Service |
| **BCE** | Before Common Era |
| **ICE** | Internal Combustion Engine |
| **SAE** | The Society of Automotive Engineers |
| **ADS** | Autonomous Driving Systems |
| **LiDAR** | Light Detection and Ranging |
| **GPS** | Global Positioning System |
| **SLAM** | Simultaneous Localization and Mapping |
| **AI** | Artificial intelligence |
| **IoT** | Internet of Things |
| **BaaS** | Backend-as-a-Service |
| **RTK** | Real-Time Kinematic |
| **MVC** | Model-View-Controller |
| **UKF** | Unscented Kalman Filter |
| **IMU** | Inertial Measurement Unit |
| **GSM** | Global System for Mobile Communications |
| **ETA** | Estimated Time of Arrival |
| **ROS** | Robot Operating System |
| **V2X** | Vehicle-to-Everything |
| **V2V** | Vehicle-to-Vehicle |
| **V2I** | Vehicle-to-Infrastructure |
| **V2P** | Vehicle-to-Pedestrian |
| **V2N** | Vehicle-to-Network |
| **WIP** | Work in Progress or Work in Process |
| **XP** | eXtreme Programming |
| **TDD** | Test-Driven Development |
| **RAD** | Rapid Application Development |
| **OSM** | OpenStreetMap |

| | |
|---|---|
| **UDP** | User Datagram Protocol |
| **API** | Application Programming Interface |

# List of Figures

# List of Tables

# Chapter 1

# Introduction

Once considered a revolutionary tool, technology outgrew its role as a new, separate idea, and slowly embedded into every aspect of life. For many, the use of mobile apps became part of their daily routine, such as getting coffee or going for a morning run. The allure and convenience of having the world at their fingertips increased daily mobile usage to an average of 2 hours and 51 minutes, and it is looking to increase to 3+ hours this year [1]. These apps are not just for entertainment; they are the interface through which people manage everything from banking and education to health and transportation.

One of the main sectors impacted by the rise of mobile applications is transportation. Apps like Uber, Careem and Go-Bus have changed how people access mobility, offering real-time, on-demand transport solutions. One of the other main sectors is education, where apps now provide access to online courses, digital libraries for books and research and interactive learning platforms. Learning is made more fun and accessable to everyone. The health sector as well has been revolutionalized, with apps encouraging individuals to track their fitness, book doctor appointments from home, manage chronic diseases and even consult with healthcare professionals remotely. Financial sector as well, with banking apps gaining popularity. From ordering groceries at home, to managing smart devices, the role of mobile applications cannot be undermined.

## 1.1 Evolution of Mobility

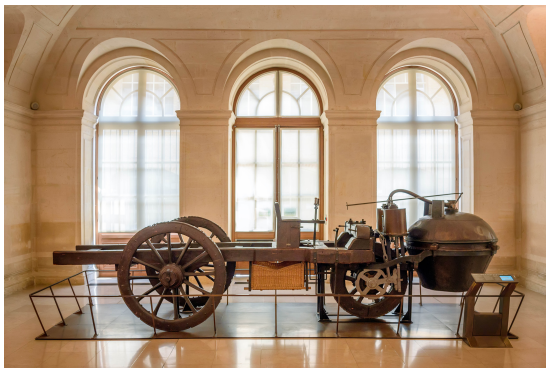
### 1.1.1 History of Transportation

The history of transportation reflects humanity's continuous drive to improve mobility. Humans decided to cross water first, making the boat the first mode of transportation created, this was estimated to be 60,000–40,000 years ago. Afterwards, with the creation of the wheel, horse-drawn carts and chariots were introduced, this dates to around 4000 Before Common

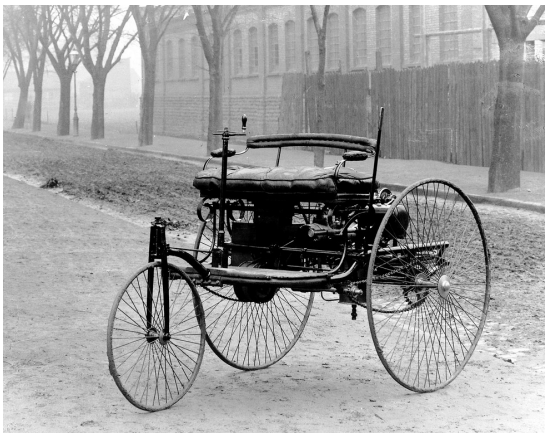
Era (BCE). The Industrial Revolution brought significant advances, particularly with the invention of the steam engine. In 1769, the Watt steam engine was created. Shortly after, the world's first successful steam ship was invented by Claude de Jouffroy in 1783 [2].

### 1.1.2 The Birth of Automobiles

The development of the steam engine as mentioned in section 1.1.1 also marked a key milestone, this was one of the turning points for automobile evolution. Although Isaac Newton and Leonardo da Vinci started the early concepts of self-propelled vehicles, the first steam-powered automobile capable of human transportation was built by Nicolas-Joseph Cugnot in 1769 as can be seen in figure 1.1a [3]. By the early 19th century, innovation shifted toward more efficient power sources [4]. Steam engines, while revolutionary, were large, heavy, and limited in speed and range. And this shift is what motivated the invention of the internal combustion engine.It kept evolving till engineers created a 4-cycle combustion engine and started using gasoline. The Benz Patent-Motorwagen, first marketable automobile powered by a gasoline engine, was created by Carl Benz in 1885 [5]. It is shown here in Figure 1.1b.



(a) First steam-powered automobile [3]



(b) Benz Patent-Motorwagen [5]

Figure 1.1: Early milestones in automobile development

### 1.1.3 Industrial Acceleration and War-Time Innovation

The two World Wars significantly accelerated automotive innovation. Wartime demand led to improvements in fuel efficiency, engine durability, and mass production techniques, and as steam engines were bulky and heavy, they became impractical and Internal Combustion Engine (ICE)s took dominance. They became the new standard, achieving a much higher horse power, and that was the most important thing back then. However, due to the oil crises of 1970, a demand for more fuel-efficient vehicles was in place. Due to that demand, engineers started creating smaller cars the used less fuel. Aerodynamics also played a significant role in

improving fuel efficiency. In the 1980s, car manufacturers started designing cars with sleeker shapes and reduced drag. One of the most popular ones was DeLorean DMC-12. Figure 1.2 is an example poster from the time.

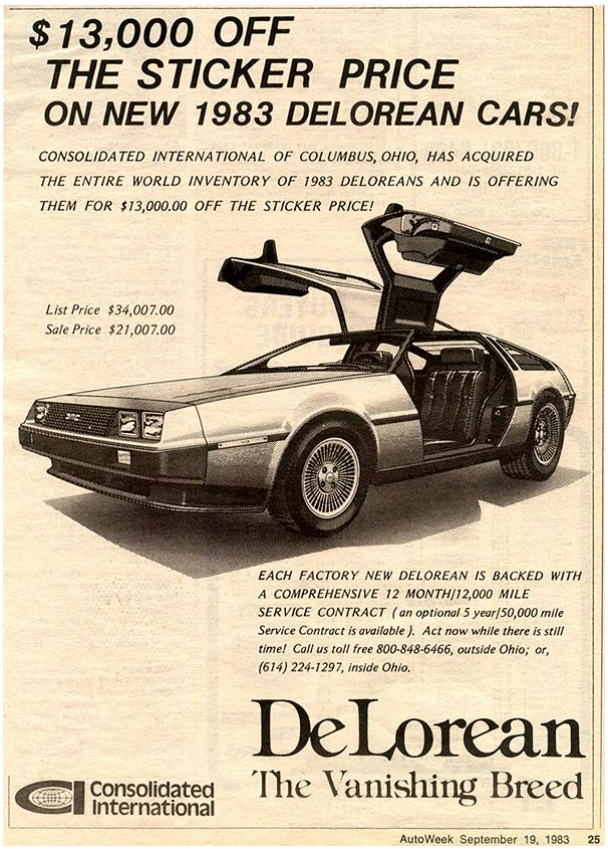



Figure 1.2: Poster of the DeLorean DMC-12[3]

### 1.1.4 Hybrid Vehicles and Their Breakthrough

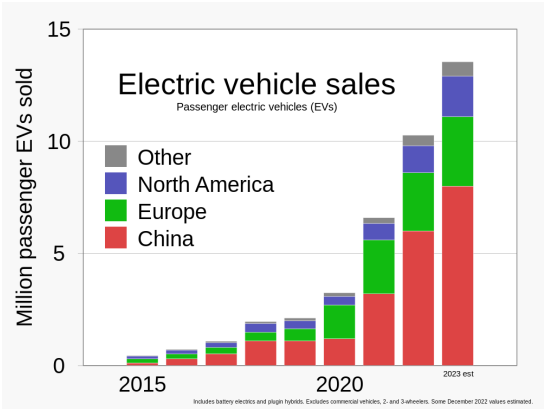
The hybrid engine was first created by the austrian engineer Ferdinand Porsche in 1900, but it did not gain popularity as ICE was dominating the market, and petrol was cheap. As fuel prices increased, demand for more efficient engines increased, and finally in 1997, the infamous Toyota Pirus was launched. It became the first mass-produced hybrid vehicle, combining a petrol engine with an electric motor to improve fuel efficiency [6].

### 1.1.5 Electric Vehicles Evolution

Electric car concepts and trials have taken place longer than expected. Robert Anderson is often credited with inventing the first electric car some time between 1832 and 1839. More trials and experiments took place in the 1880s. But they lost popularity due to ICE vehicles being in high demand at the time. In the early 1990s, California Air Resources Board sought after

low-emission vehicles, and car manufacturers started exploring electric vehclies again. Tesla Roadster would be the first highway-legal all-electric car to use lithium-ion battery cells, and the first production all-electric car to travel more than 320 km per charge. Other manufacturers started releasing models as well, such as the Mitsubishi i-MiEV, launched in 2009 in Japan. A few of months later the Nissan Leaf, surpassed the i-MiEV as the best selling all-electric car at that time. Tesla's Model S , in Figure 1.3a, was awarded the title "ultimate car of the year" in 2019. In March 2020 the Tesla Model 3 became the world's all-time best-selling electric car, with more than 500,000 units delivered. In recent years, the electric vehicle market has grown rapidly, by the third quarter of 2021 Electric Vehicles (EVs) accounted for 6% of all U.S. light-duty vehicle sales, with 187,000 units sold, a high record and 11% increase compared to a 1,3% increase in gasoline and diesel vehicles [7]. Figure 1.3b shows the rise in sales of EVs.

(a) Tesla Model S [8]




(b) Sales of electric vehicles [7]

Figure 1.3: Electric vehicle example and sales

## 1.2 Autonomous Vehicles

### 1.2.1 Introducing Autonomy

What does autonomy mean? In the Cambridge Dictionary it is defined as "the right of an organization, country, or region to be independent and govern itself" [9].The term is derived from the ancient Greek words **autos**, which means "self," and **nomos**, meaning "rule" [10]. To simply put it, it is to be able to make your own decisions without being controlled by anyone else. The Society of Automotive Engineers (SAE) has developed a widely recognized standard that classifies road vehicle autonomy into six levels,from 0 to 5, based on the degree of driver intervention and system capability, it will be discussed in detail in Chapter 2.

### 1.2.2 Brief History of Autonomous Vehicles

The concept of autonomous vehicles dates back to the 16th century when Leonardo da Vinci designed a self-propelled cart. Years later in 1925, electrical engineer Francis P. Houdina radio-controlled a full-size automobile through the streets of New York [11]. Later, researchers at Stanford University developed a small cart to navigate on the surface of the moon using a basic form of computer vision, this was in 1961. Mercedens Benz in the 80s designed a robotic van that achieved a speed of 95.9 km/h on streets without traffic. In the same decade, the DARPA-funded Autonomous Land driven Vehicle (ALV) was the first project to use lidar, computer vision and autonomous robotic control to control an AV and reach speeds up to 30 km/h. In 1995, Carnegie Mellon University's Navlab project completed a 5,000 km journey where 98.2% of it was autonomously controlled, it was called "No Hands Across America".However, this car was semi-autonomous, a level 1 or 2 on the SAE autonomy scale. Fast-forward to the 2010s, any major automotive manufacturers, including General Motors, Ford, Mercedes-Benz, Volkswagen, Audi, Nissan, Toyota, BMW, and Volvo, are in the process of testing self-driving car systems.In January 2014, Induct Technology's Navia shuttle became the first self-driving vehicle to be available for commercial sale. Later, Tesla Motors released its first version of Autopilot Model S cars with a system that is capable of lane control with autonomous steering, braking, and speed limit adjustment based on signal image recognition [12]. Figure 1 is a timeline of the most significant AVs.

### 1.2.3 Why Autonomous Vehicles Matter Today

One of the most important reasons so much funding and research is going into AV development, is because of roadway safety. In 2023 statistics show that just in the United States, more than 40,000 people die annually as a result of motor-vehicle collisions. Because more than 90% of collisions are the result of human error, Autonomous Driving Systems (ADS) technology presents an enormous opportunity for safety improvement [13]. Furthermore, AVs will help decrease transportation costs. When companies scale the ADS technology, they will be able to provide ride-hailing and freight trucking at lower prices. ADS may soon be able to reduce ride-hailing fare prices to half its original cost and reduce trucking costs by 30% [13]. Autonomous Vehicle (AV)s will also allow accessibility and independence to individuals who are unable to drive or use public transport due to age, disability or other factors [14]. Additionally, the environment will be the greatest beneficiary. With Autonomous Vehicle (AV)s, there are less accidents, therefore less road congestion and traffic, resulting in reducing fuel usage and carbon emissions [15].

## 1.3 Mobility Applications

Mobility applications are digital platforms that enable users to access, order, plan, and pay for transportation services using their smartphones or other devices. These apps have transformed how people interact with transportation systems by offering real-time tracking, instant booking, dynamic pricing, and seamless payments. Their role is the core of the Mobility as a Service (MaaS) concept, which aims to integrate multiple transportation services into one ecosystem for convenience.

### 1.3.1 Mobility-as-a-Service

As mentioned above in section 1.3, MaaS is the future of urban mobility. A user can book a ride, ride-share or rent a bike/scooter all in one app. Cars are not the only variable, buses, bikes, scooters, trains and metros are all being slowly included. The integration is to simplify finding, booking and paying for transportation, by having it all in one place. Furthermore, MaaS is trying to reduce the people's reliance on private transport (owning a car), which could significantly decrease traffic congestion, carbon emissions, and transportation costs. It also has potential to make transport more affordable for low-income individuals/families.In the current economy buying and maintaining a car is quite expensive, especially with rising petrol prices, so ride-sharing apps are a great affordable solution [16].

### 1.3.2 Brief History of Mobile Apps

Mobile applications became an integral part of our lives, but where did it all start? In 1994, IBM launched Simon, the first smartphone. It contained some simple applications like calendar, notes, calculator etc. In 1997, Nokia included the ”snake” game in its phone. In 2002, RIM launched the Blackberry 5810 with pre-loaded apps, also introducing wireless-email. However, the pivotal point was when Apple released the Iphone in 2007, and later in 2008 released the Apple store in an OS update, with over 500 apps. 10 million downloads took place within 3 days of the release [17].

### 1.3.3 Rise of Ride-Hailing Apps

It all began with George Arison, who after being sent by his company to attend meeting in different cities, realised how tiring and hard it is to hail taxis in new cities. After drinks with his friend Toby Russell and entrepreneur Tom Depasquale, they came up with RideCharge. It launched in December, with the name TaxiMagic and received over 30,000 downloads in a day, but the success did not last. There were multiple issues in the business, main ones being the long and complicated payment process and flaws with the booking system [18]. RideCharge was the blueprint, but Uber was revolutionary. It was released in 2011 and today it is the largest ride-sharing company worldwide with over 150 million monthly active users and 6 million active

drivers. It coordinates an average of 28 million trips per day, and has coordinated 47 billion trips since its release [19]. This real-time, GPS-enabled model soon inspired global competitors such as Lyft (USA), Bolt (Europe), Didi (China), Careem (India) and Grab (Southeast Asia). These apps offered a reliable alternative to traditional taxis, providing estimated arrival times, digital payments, and customer reviews. Their impact was significant and it reshaped urban mobility [19].

### 1.3.4 Mobility Apps for Autonomous Vehicles

As autonomous vehicles rise, mobility apps have evolved to support interaction with these self-driving cars. Google's development of self-driving technology began in January 2009 [20]. After two years of road testing, the project was revealed in October 2010. In 2015, the world's first fully autonomous ride on public roads was taken by Steven Mahan, who was legally blind [21]. In October 2020, Waymo became the first company to offer service to the public without a driver in the vehicle, the app was released on Google Play and the App Store so anyone in the U.S could download it and order a ride. Baidu, a company based in China, launched Apollo Go an autonomous ride-hailing service operating in cities like Beijing and Wuhan [22]. Zoox, created by Amazon, is developing autonomous vehicles designed for ride-hailing, with plans to launch services in select cities [23].

### 1.3.5 University Campus Context

If ride-hailing and sharing are beneficial and important in cities, then it is crucial on college campuses. Students face multiple challenges, one of the main ones is the financial burden of transportation to and from campus. Studies show that transportation costs alone can be up to 20% of the total cost of attending college [24]. Additionally, 82.5% of students in the University of Alabama at Birmingham, for example,drive to campus[25], which contributes to traffic congestion on campus and leaves huge carbon footprints.

Consider, for instance, the case of a student living off-campus, in need of a daily ride. They have to choose between either unreliable public transport, walking a long and potentially dangerous distance, or face the expenses of owning and maintaining a car. Similarly for a faculty members, who circle the crowded parking lot looking for a place, while being late for an important lecture or meeting.

Beyond the commuting issue, university campuses are huge, with vast distances between classes, labs, dorms and lecture halls. Walking across these areas can be time-consuming, especially for students with disabilities, tight class schedules, or during poor weather conditions. Hence, university campuses are an ideal setting for implementing mobility solutions. An autonomous shuttle or on-demand in-campus ride-hailing apps would be highly beneficial.

Some universities already started exploring and implementing similar concepts. In 2016,

University of Michigan introduced NAVYA, a fully autonomous 15-passenger shuttle, used for campus tours [26]. Moreover, in 2021, May Mobility provided autonomous rides on University of Texas at Arlington campus and downtown area. Since then, the service has provided over 100,000 rides for student and faculty members [27].

## 1.4 Problem Statement

As transportation technologies rapidly evolve, university campuses continue to rely on outdated, inefficient, and expensive mobility systems that are unable to meet the needs of students and staff. AVs and mobility applications are gaining significant popularity these days, however, their adoption in campus environments remains extremely limited.
Current ride-hailing services are not optimized for in-campus travel, nor do they integrate AV technology. There exists a gap in developing a platform dedicated for in-campus ride-hailing services. This thesis addresses that gap by exploring the design and implementation of a ride-hailing application specifically developed for AV-based mobility in university campuses.

## 1.5 Thesis Organization

This thesis is structured into the following chapters:

- **Chapter 1 – Introduction:** Provides background on the evolution of transportation and mobility, the rise of autonomous vehicles (AVs), and the motivation for developing an ride-hailing application for university campuses.
- **Chapter 2 – Literature Review:** Reviews related work in AV localization, ride-hailing systems, and mobile application development. It highlights current technologies, frameworks, and gaps in existing research.
- **Chapter 3 – Methodology:** Describes the technical design and development of the Go-Drives application. It includes system architecture, software tools, AV communication, localization methods, development workflow, and simulation setup.
- **Chapter 4 – Results:** Presents testing results from GPS accuracy experiments, ride simulation trials, notification delivery analysis, camera integration, and user feedback on app usability. This section also includes the challenges encountered during the project.
- **Chapter 5 – Conclusion:** Summarizes the project's outcomes, reiterates the objectives, and reflects on the overall success and impact of the thesis work.
- **Chapter 6 – Future Work:** Outlines potential enhancements for the application and AV integration.

# Chapter 2

# Literature Review

## 2.1 Literature Summary

As discussed previously, the goal of this thesis is to develop a ride-hailing application specifically designed for autonomous vehicles operating within a university campus environment. Before starting the design and implementation phases, it is important to research and understand the key foundation areas this project is built on. This literature review aims to identify best practices, existing solutions, and current research gaps.

Key areas of research include mobile application development, focusing on frameworks and platforms. Existing implementations of autonomous vehicles on university campuses will also be analyzed. By analyzing similar AV projects, this review highlights the technological approaches adopted, their limitation and any areas for future improvement.

Additionally, communication protocols are examined to compare methodologies used in AV systems, mainly focusing on real-time data exchange and system integration. The review also explores localization techniques.

## 2.2 Autonomous Vehicle Systems and Technologies

With electric mobility gaining widespread adoption, the next step in transportation technology is the AV, which aims to eliminate human driving altogether. Vehicular automation is the use of technology to help or fully replace the operator of a car, truck, aircraft, rocket, military vehicle, or boat [28].

### 2.2.1 SAE Autonomy Levels

The Society of Automotive Engineers have developed a widely recognized standard that classifies road vehicle autonomy into six levels, from 0 to 5, based on the degree of driver intervention and system capability [29]:

**Level 0: No automation**
Driver is responsible for all the driving tasks, however there are very limited features like warning or automatic emergency braking.

**Level 1: Driver Assistance**
System can assist with some tasks like steering **OR** acceleration/brake support, not both.

**Level 2: Partial Automation**
System can assist with both steering **AND** acceleration/brake at the same time.

**Level 3: Conditional Automation**
The vehicle can perform all driving tasks in specific conditions, but the human driver must be available to take over when requested.

**Level 4: High Automation**
Driver will not be requested to take over, but the automation is conditional (available for limited conditions, will not operate unless conditions are met).

**Level 5: Full Automation**
Vehicle will operate under all and any conditions and the driver will not be asked to take over.

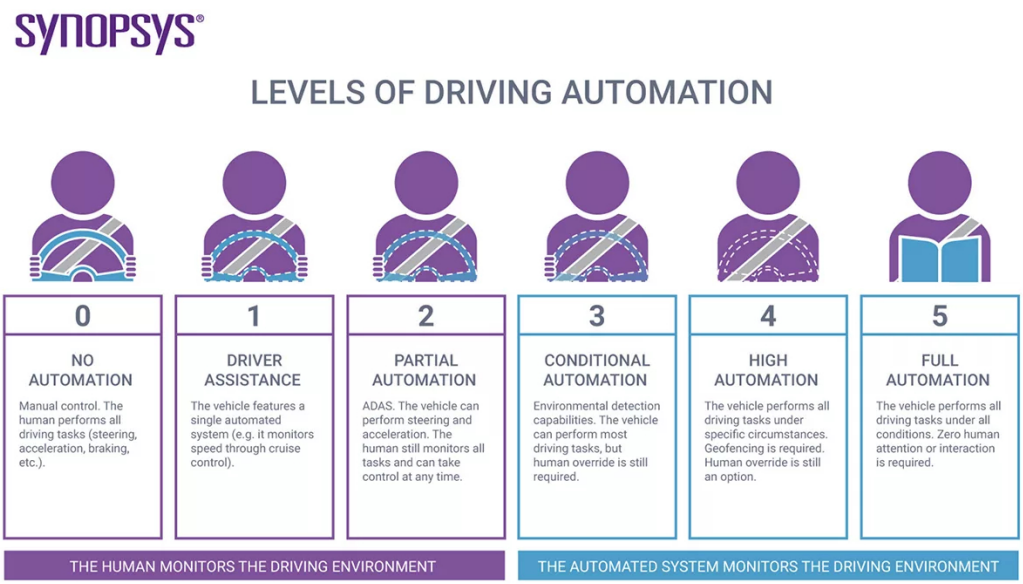



Figure 2.1: Autonomy Levels [30]

### 2.2.2 Core Technologies Behind Autonomous Vehicles

To understand AVs and how they work, a research was conducted on them and the core technologies they need to work. **Sensors and Cameras** The sensory system of an AV is essential for its functionality. These vehicles are equipped with various types of sensors, including Light Detection and Ranging (LiDAR), radar, and ultrasonic sensors. LiDAR uses laser beams to create a detailed 360 degree 3D map of the surroundings, and the radar helps in detecting the speed and movement of objects. Ultrasonic sensors are used for close-range detection, like during parking maneuvers. Cameras provide visual information, allowing the vehicle to recognize traffic signs, road markings, pedestrians, and other vehicles on the road [31][32].

The perception system collects and processes data from all these sensors to understand the vehicle's environment. This involves object detection, classification and tracking algorithms to identify and monitor obstacles, road conditions, neighboring vehicles and street-crossing pedestrians [33].

**Localization and Mapping** High-definition maps and Global Positioning System (GPS) technologies allow AVs to determine their precise location and plan routes effectively. For road vehicles, there are two main approaches. Vehicles either have basic maps and depend on real-time mapping techniques like Simultaneous Localization and Mapping (SLAM) for

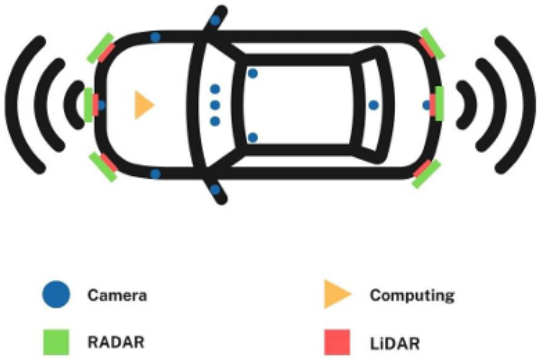



Figure 2.2: Sensor Types and Positions on AVs [31]

navigation, these rely heavily on their perception system. On the other hand, some vehicles use highly detailed pre-mapped data that reduces the need for real-time decision-making. However, this approach needs constant maintenance, as the environment continuously changes [34][28].

**Software and AI** The core of AV technology, is Artificial intelligence (AI). AI algorithms, in particular the machine-learning-based ones, are trained on hundreds of data sets with different scenarios, so the AV can handle any complex situation quickly and accurately[33].

**Decision-Making and Planning** Autonomous vehicles use control algorithms to operate safely. These algorithms receive data from the perception system and GPS, and make decisions on acceleration, braking and steering[34].

Figure 2.2 represents the positioning of different sensors on a vehicle.

### 2.2.3 Localization in Avs and Mobile Applications

As mentioned above, accurate localization is a fundamental requirement for the safe and efficient operation of autonomous vehicles, especially in ride-hailing services where exact pickups and drop-offs are essential. As localization is one of the key components of this project, further research was conducted on various localization methods, such as GPS-based tracking, sensor fusion techniques, and mobile app-based location sharing.

#### Limitations of GPS and Enhancements with RTK

The most commonly used localization method in AVs is the GPS, which typically offers position accuracy ranging from 0.5 to 3.5 meters [35]. It can be further enhanced to centimeter-level accuracy with Real-Time Kinematic (RTK) technology, but even the performance of RTK-GPS is restricted due to weather conditions and the jumping/drifting that occurs in obstructed areas like dense urban areas, in tunnels, or under bridges.

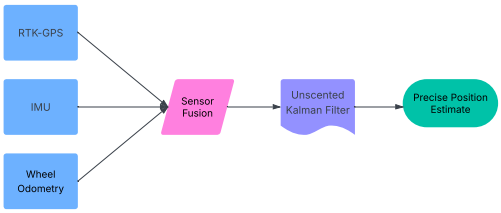



Figure 2.3: Sensor Fusion Pipeline combining RTK-GPS, IMU, and Wheel Odometry for precise vehicle localization using a UKF.

### Multisensor Fusion for Campus AV Positioning

Due to the RTK-GPS limitations, a research project that was researching the best practices for vehicle positioning of an autonomous vehicle prototype in a campus environment, combined wheel odometry, Inertial Measurement Unit (IMU)s and RTK-GPS sensor data to create a stable and accurate multisensory, fusion-based vehicle positioning system. The Unscented Kalman Filter (UKF) was applied to this fused data to obtain the most accurate results. When travel distances of a trip where obtained from RTK-GPS, wheel odometry and UKF sensor fusion approach, UKF was the most accurate with a 0.8% error [35].

### Layered Localization Methods for Taxi AVs

In a study developing an autonomous taxi service for a university campus, Kim et al. [36] identified that standalone GPS accuracy was insufficient for the vehicle's localization needs, especially in areas with weak and unreliable satellite signals. To ensure accurate positioning across different road types and environmental conditions, the researchers implemented a hybrid approach that used three distinct localization techniques.

First, in complex or areas with weak GPS signals, such as urban canyons or tunnels, they employed a **3D map-based localization method**. This technique used 3D LiDAR-generated occupancy grid projected onto a 2D plane, combined with particle filtering, to provide precise vehicle positioning even in the absence of real-time GPS data.

Second, for structured roads equipped with lane-level maps, a **map-matching approach** was applied. This involved using spline-modeled lane data and aligning it with lane markings detected by onboard cameras, GPS, and motion sensors, they used the particle filter again to estimate the vehicle's position.

Lastly, for **well-structured roads without access to a map**, a vision-only lane detection system was employed. By analyzing visual lane features through probabilistic models and algorithms, the system could estimate the ego vehicle's position in the lane in real-time.

This multi-layered localization strategy ensured reliable and adaptable positioning performance across a variety of campus road conditions.

#### GSM-GPS Tracking in Theft and Emergency Systems

A GPS-based tracking systems have also been proposed for vehicle theft prevention. In a project, the researchers proposed a tracking system for stolen vehicles, it used GPS and Global System for Mobile Communications (GSM) [37]. The GPS-GSM based tracking device can be controlled through the use of text messages. When the user sent a message to the system, the coordinates of the vehicle would be sent back. An accuracy test was conducted in areas with constraints (such as bad weather, enclosures, and high rise buildings) and areas without. The variation in distance accuracy in unobstructed areas was 0.6 m and 0.9 m, while in obstructed areas it got up to 5 m.

Furthermore Virginus et al. [38], created a very similar system to the one mentioned above, where a mystery button was installed in the vehicle, and after starting the car, if the button was not pressed within a 40 second time span, an SMS was sent to the car owner with the exact Global Positioning System (GPS) coordinates of the car.

#### GPS-Based Bus Tracking Applications

Several studies explored GPS-based tracking for public transportation. In one system, GPS data including bus speed and Estimated Time of Arrival (ETA) was sent via GSM to a server for real-time access by users and administrators [39]. Another system by Akter et al. [40] used a dual-app model: the driver's app shared location from their smartphone GPS, while the user app displayed real-time bus information such as location, routes and available seats. Similarly, Narasimhan et al. [41] used the driver's phone GPS to update the vehicle's location in the system without the need for external hardware.

### 2.2.4 Communication in Autonomous Vehicle Systems

Effective communication is essential for AVs to interact with their environment, infrastructure, other vehicles, and cloud systems. This communication enables perception sharing, collaborative decision-making, coordinated movement, and real-time system updates. AV communication can be broadly classified into several types, each serving a distinct purpose within the transportation ecosystem.

#### Vehicle-to-Everything (V2X) Communication Technologies

V2X communication has several subcategories:

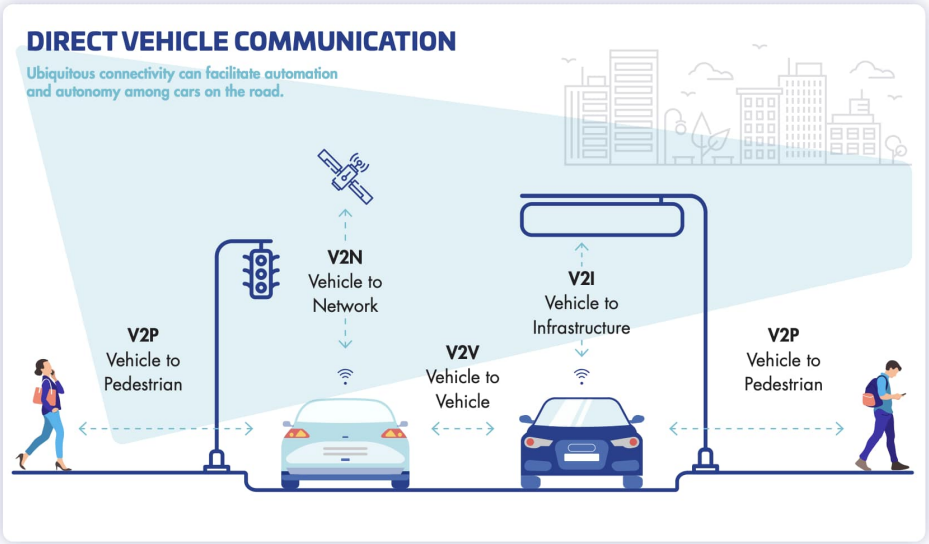



Figure 2.4: Vehicle-To-Everything [42]

- **Vehicle-to-Vehicle (V2V)** Direct communication between vehicles to share speed, position, and trajectory to avoid collisions and improve traffic flow.
- **Vehicle-to-Infrastructure (V2I)** Communication with traffic signals, road signs, and smart infrastructure to optimize routing and comply with local rules.
- **Vehicle-to-Pedestrian (V2P)** Alerts and interactions with nearby pedestrians via mobile devices or wearable technology.
- **Vehicle-to-Network (V2N)** Internet/cloud communication for updates, remote control, or fleet management.

These communication types are visualized in Figure 2.4.

### 2.2.5 Communication Protocols and Channels

To enable reliable and real-time data exchange, AVs rely on multiple communication protocols and transmission channels. The selection of communication method depends on use case, range, latency, and bandwidth needs.

- **Wi-Fi:** Common for high-bandwidth short-range communication, especially for V2I and internal system diagnostics.
- **Bluetooth:** Suitable for short-range, low-latency tasks like connecting to mobile devices or wearables.
- **GSM/LTE/5G:** Cellular networks are used for V2N communication and enable remote

fleet monitoring, **OTA!** (**OTA!**) updates, and cloud services.

- **UDP/TCP:** These transport layer protocols are widely used in AV systems. UDP is preferred for fast, real-time broadcasting (e.g., video streams), while TCP is used for reliable transmission where order and completeness matter, like for sensor data.

Many AVs integrate multiple protocols simultaneously, prioritizing speed, security, and redundancy in mission-critical tasks.

Table 2.1: Comparison of Communication Protocols in AV Systems

| **Protocol** | **Range** | **Latency/Bandwidth** | **Common Use in AV Systems** |
|---|---|---|---|
| Wi-Fi | Up to 100 m | Low latency, high bandwidth | Short-range data sharing, V2I, local diagnostics |
| Bluetooth | 10 m | Very low latency, low bandwidth | Sensor pairing, V2P, wearable/personal device links |
| GSM/LTE/5G | Wide-area (km) | Medium–high latency, moderate–high bandwidth | V2N, OTA updates, cloud services, remote monitoring |
| UDP | Dependent on network | Low latency, unreliable | Real-time streaming (e.g., LiDAR, camera data) |
| TCP | Dependent on network | High reliability, higher latency | Logging, file transfer, control messages |

### Robot Operating System (ROS)

The ROS is a widely adopted open-source middleware framework used for communication and modular development in robotics and AVs. ROS organizes system functionality into distributed nodes that communicate through message-passing using a publisher/subscriber model or service calls.

**Node-Based Architecture and Communication** ROS is built on a distributed node-based architecture, where each software module or process (called a node) performs a specific function, such as sensor data collection, localization, or path planning. These nodes communicate via a publisher/subscriber model using topics, or via request/response communication using

services. For example, a node subscribing to the /gps topic might receive latitude and longitude data published by a GPS node. This distributed/decoupled architecture supports high modularity, allowing different parts of the system to be developed, tested, maintained, and replaced independently.

**Common ROS Packages in AV Systems** A wide variety of ROS packages are commonly used in AV applications:

- `navigation`, `move_base` for path planning and movement
- `sensor_msgs` for standard sensor data formatting such as LiDar, GPS, IMU and cameras.
- `rosbag` for recording and replaying ROS messages for testing and debugging.

These packages provide a strong foundation for implementing perception, control, and decision-making systems in autonomous vehicles.

## 2.3 Software Frameworks and App Development

### 2.3.1 Front-End Frameworks

When developing a mobile application for autonomous vehicle ride-hailing, choosing the right development framework is a critical decision.

From research into app development and what similar project used, it was concluded that there are many available platforms. There are native options with Swift (for iOS) and Kotlin (for Android) which provide excellent performance, but this creates a longer development cycle, as the work has to be duplicated for cross-platform deployment [41].

In other projects, platforms such as Blynk [43], MIT App Inventor [44], and PhoneGap [39] were used. Blynk is famous with Internet of Things (IoT) projects; it simplifies the process of creating user interfaces for controlling hardware, but this functionality was not important for this project. MIT App Inventor is great for beginners and learning the basics of app development, however, it is not suitable for building complex applications. Finally, PhoneGap was popular in the earlier days of cross-platform app development, but it often faces challenges in terms of performance and user interface responsiveness compared to other recent, better platforms.

Given the goal of developing a scalable, cross-platform ride-hailing application for AVs, this study focuses on three widely used frameworks that support both iOS and Android from a single code base: **React Native**, **Flutter**, and **Xamarin**. These frameworks are evaluated based on language used, learning curve, community support, performance, development speed, and platform compatibility.

**React Native**, which was developed by Facebook, allows developers to build native mobile applications using JavaScript and React. It provides access to native components via a bridge and is supported by a large community. It is also very compatible with Expo, which makes it great for prototyping and testing.

**Flutter**, developed by Google, is a UI toolkit that uses the Dart programming language. It compiles to native code, providing excellent performance and consistent behavior across platforms. Flutter includes a large set of pre-designed widgets and supports cross-platform deployment.

**Xamarin**, supported by Microsoft, enables cross-platform mobile development using C and the .NET framework. It compiles to native code and integrates well with Visual Studio, making it ideal for developers already familiar with Microsoft's ecosystem. Xamarin offers strong performance and native API access, especially through Xamarin.Native and Xamarin.Forms. However, its smaller community and slightly limited UI flexibility make it less popular than React Native or Flutter for modern consumer-facing apps.

Table 2.2: Comparison of Selected Front-End Frameworks

| **Criteria** | **React Native** | **Flutter** | **Xamarin** |
|---|---|---|---|
| Language Used | JavaScript | Dart | C# |
| Learning Curve | Easy for Web Developers | Steep for Dart | Easy for C# developers |
| Community Support | Excellent | Growing rapidly | Moderate |
| Performance | Lowest performance with largest binary size and high memory usage | Comparable to Xamarin; better rendering but large binaries | Most efficient in CPU and memory usage; smallest binary size |
| Platform Support | iOS, Android, Web (via Expo) | iOS, Android, Web | iOS, Android, Windows |

Table 2.2 summarizes the results of multiple research projects that compared all three frameworks [45, 46, 47]. The researchers would create the same app using the different frameworks and compare performance, ease-of-use, app size, etc. All researches have concluded that there is no huge difference between the frameworks, it all depends on what the developer needs. React Native is the best for web developers and complex apps because JavaScript is widely used in web programming. Flutter is recommended for those who are familiar with C++ or Java,

because Dart is similar to both. And finally, Xamarin is more suitable for .NET developers or those who are concerned about performance.

### 2.3.2 Back-End Technologies and Databases

The back-end and database used in the project are just as important as the front-end framework. An extensive research was conducted, however, none of the papers that implemented mobile applications for AV projects provided any information on what backend they used. Therefore, only the database services will be compared.

**Firebase**, developed by Google, is a Backend-as-a-Service (BaaS) platform that provides cloud-hosted NoSQL storage in JSON format. It has many features, some of them include:

- **Authentication:** Login through Gmail, GitHub, Twitter, Facebook, and custom methods.
- **Analytics:** Gather usage data about users, active sessions, and most-used features.
- **Crash Reporting:** Automatically detects app crashes, also developers can define custom crash events.

It is a popular platform, especially in projects who need to develop an app quickly and not worry about managing seperate backend and database, like [41, 39, 40, 48].

**MongoDB**, similarly to Firebase, is a NoSQL, open-source document-oriented database. It offers high performance and easy scalability. It supports real-time features and is often used with Node.js or other server-side platforms.

The key differences and similarities between Firebase and MongoDB are summarized in Table 2.3 [49].

Table 2.3: Comparison of Firebase and MongoDB for Real-Time Applications

| **Criteria** | **Firebase** | **MongoDB** |
|---|---|---|
| Type | Backend-as-a-Service (BaaS) | NoSQL document store |
| Real-time Capability | Built-in real-time sync across all clients | Supports real-time |
| Data Format | JSON | BSON (Binary JSON) |
| Authentication | Integrated with Google, GitHub, Twitter, etc. | Requires custom implementation |
| Hosting | Cloud-hosted by Google | Self-hosted or cloud |
| Scalability | High, managed by Google | High, user-managed via clusters |
| Pricing | Free tier available, pay-as-you-go | Free |
| Ease of Setup | Very easy | More setup required, especially with backend frameworks |

### 2.3.3 Software Development Methodologies

Software development methodologies are structured approaches used to design, develop, and test software applications. These methodologies help align development with project goals, improve quality, and reduce cost [50]. This section provides a brief overview of popular methodologies and their suitability for different projects.

#### Agile Development Methods

Agile is an iterative and incremental development approach that mainly focuses on flexibility, collaboration, and continuous customer feedback. Work is organized into short cycles called sprints.

**When to Use:** Ideal for projects with evolving requirements or high user involvement.

**Scrum** Scrum is the most popular agile framework [51]. It divides work into time-boxed sprints (1–4 weeks) and includes defined roles (e.g. Product Owner, Scrum Master) and events (e.g. Daily Scrum, Sprint Review).

**Kanban** A visual workflow management system using boards and cards to track tasks. It limits Work in Progress or Work in Process (WIP) and is best for continuous delivery and support [52].

**eXtreme Programming (XP)** Focuses on engineering excellence through practices like pair programming, Test-Driven Development (TDD), and continuous integration. Ideal for high-risk or fast-changing environments [53].

#### Traditional Development Methods (Linear & Sequential)

Traditional methods follow a step-by-step process where each phase must be completed before moving to the next. These methods prioritize upfront planning, documentation, and formal control.

**When to Use:** Suitable for stable, well-defined projects or regulated environments where documentation is critical.

**Waterfall Model** A classic linear model with sequential phases: Requirements → Design → Implementation → Testing → Deployment. Best for small, predictable projects [54].

**Spiral Model** Combines iterative development with risk assessment. Each loop involves planning, risk evaluation, and development. Best for large, high-risk projects [55].

**Rapid Application Development (RAD)** Focuses on quick delivery through prototyping and user feedback. Best for projects with low technical risk and need for rapid iterations [56].

## 2.4 Autonomous Vehicles in Mobility Services

### 2.4.1 Ride-Hailing Apps for Autonomous Vehicles

With self-driving cars moving from research labs to real-world deployment, the most critical development area is integrating them into ride-hailing. Such systems require not only advanced vehicle autonomy but also ease-of-use and simple interfaces to reserve, track, and navigate rides through mobile apps. A few companies have already launched commercial or semi-commercial AV ride-hailing offerings.

#### Waymo One

Waymo is widely considered the front-runner in autonomous ride-hailing. Waymo launched Waymo One, the world's first commercial robotaxi service, which first hit Phoenix, Arizona's roads in 2018. Users can hail rides using the Waymo app, which functions much like Uber or

Lyft, but without a driver behind the wheel. The vehicles, which are mainly modified Chrysler Pacifica hybrids and Jaguar I-PACE EVs, operate at SAE Level 4 autonomy within a well-mapped geofenced area. The app enables real-time tracking, ETA, and even in-car controls for locking doors or adjusting climate settings. Waymo has since expanded its service to include parts of San Francisco and Los Angeles. While some rides still include safety drivers, fully driverless rides are becoming more common as the system matures [57].

#### Cruise

General Motors funded Cruise, which is another major competitor in the autonomous ride-hailing space. In 2022, Cruise launched its fully driverless ride-hailing service to the public in San Francisco, using a fleet of Chevrolet Bolt EVs. The Cruise mobile app allows users to order and monitor AVs, though the service initially operated only during nighttime hours and in specific areas of the city. Unlike Waymo's slower, more cautious approach, Cruise's strategy focuses heavily on urban deployment and dense traffic environments, which presents unique technical challenges. Cruise has also announced plans to expand into other cities like Austin and Phoenix. The service highlights sustainability by using an electric-only fleet and aims for a highly scalable model that integrates AVs into existing urban transit systems and transportation networks [58].

#### Other Emerging Services

Other companies are also entering the autonomous ride-hailing space. In China, Baidu's Apollo Go is one of the largest AV ride-hailing service, operating in cities like Beijing, Wuhan, and Chongqing [59]. It also uses a separate app to request rides and is licensed to offer fully autonomous service in several districts. Similarly, Motional partnered with Lyft to launch driverless ride-hailing pilots in Las Vegas, with riders able to request AVs through the Lyft app interface [60]. These projects represent the global trend towards scalable, app-based AV ride-hailing, but most are so far restricted to highly controlled inner cities.

### 2.4.2 Autonomous Vehicles in Campus Environments

As previously mentioned, autonomous vehicles have seen a growing interest, especially in controlled environments. And especially for this project, the review of related works will focus on AVs on campuses. Several institutions worldwide have begun deploying AVs for transportation, delivery, or research purposes. These projects vary in vehicle type, access method, and intended use, from fully operational shuttles for students to experimental platforms for robotics research.

This section highlights selected university AV deployments that are diverse in geography, function, and technological approach, providing valuable context. A summary of these projects

is provided in Table 2.4.

Table 2.4: Autonomous Vehicle Projects at Selected Universities

| University | Country | Vehicle Type | Access Method | Project Description and Link |
|---|---|---|---|---|
| **University of Michigan** | USA | Autonomous Shuttle | App-based + Stops | Operated by May Mobility; offers 18 stops in Ann Arbor. Used for research and public transport. [61] |
| **Mississippi State University (MSU)** | USA | Autonomous Shuttle | Designated Stops | Campus Autonomous Bus (C.A.B) service — the first in the SEC. Operates on two main campus routes. [62] |
| **University of Waterloo** | Canada | Autonomous Shuttle | App + Stops | 2.7 km AV shuttle route; Canada's first public autonomous shuttle on campus using Rogers 5G. [63] |
| **King Abdullah University of Science and Technology (KAUST)** | Saudi Arabia | Autonomous Shuttle (Olli) | Designated Stops | 1.9-mile loop with 4 loading zones. Olli shuttle used to connect core campus buildings. [64] |
| **University of Western Australia (UWA)** | Australia | Autonomous Shuttle (student-built) | Designated Stops | UWA students built the first fully autonomous shuttle control system in Australia. [65] |
| **Karlsruhe Institute of Technology (KIT)** | Germany | Delivery Robots | App-based (parcel collection) | Uses autonomous delivery robots for last-mile logistics on campus via efeuCampus project. [66] |

## 2.5 Summary of Related Works

Although no single study directly mirrors the scope of this thesis, a mobile ride-hailing application designed for autonomous vehicles on a university campus, numerous research papers address individual components that are integral to the system. These include localization tech-

niques, communication protocols, app development platforms, and AV integration in controlled environments. This section summarizes selected works, identifying their use cases, methodologies, and relevance to different aspects of the current project.

Table 2.5: Summary of Related Works

| **Reference** | **Localization Method** | **Vehicle Type** | **Project Type** | **App Used** | **Tech Stack (if app)** | **Communication Method** |
|---|---|---|---|---|---|---|
| [35] | RTK-GPS + IMU + Odometry with UKF | Campus AV prototype (Electric Vehicle) | Localization system for autonomous vehicle | No | N/A | Sensor fusion |
| [36] | Not specified | Autonomous Taxi | AV Taxi campus service | No | N/A | Not specified |
| [41] | GPS (driver phone) via Google Maps API | Shuttle Bus | Real-time Bus tracking app | Yes | Android Studio (Android) + Firebase | GSM (smartphone) / Data transmission to server |
| [40] | GPS + RFID | Public Buses | Transport tracking app (with RFID) | Yes | Android Studio + Firebase | GSM |
| [37] | GPS | Standard Vehicle | Theft tracking system | Yes (via SMS) | Arduino, SIM800L, NEO-6M GPS | GSM SMS |
| [38] | GPS + GSM | Standard Vehicle | Vehicle Tracking System | Yes (SMS alert) | GSM module | GSM (SMS) |
| [48] | GPS + Zigbee | Bus Tracking | Smart Transportation System / Vehicle Tracking | Yes | Firebase | Zigbee, Wi-Fi |
| [39] | GPS (hardware) | Bus | Real-time system for users/admins | Yes | PhoneGap + Firebase | Firebase (Cloud Database) |
| [44] | Ultrasound sensor + ESP32 (obstacle detection) | Walking Stick for Visually Impaired | IoT-Based Obstacle Detection System for Visually Impaired Person | Yes (Android app) | MIT App Inventor | Bluetooth (between ESP32 and smartphone) |
| [43] | N/A | N/A | IoT Based Smart Home | Yes (smartphone app via Blynk) | Blynk framework | Wi-Fi |

## 2.6 Research Gaps

While commercial ride-hailing applications such as Uber and Careem are widely used around the world, and even autonomous ride-hailing platforms such as Waymo One and Cruise exist, there is a noticeable lack of academic literature or open-source documentation that thoroughly explores the backend logic, system design, or architecture of such applications.

As highlighted in 2.5, most existing papers focus on individual subsystems like localization, GPS tracking, or vehicle communication. However, there is limited research that combines these into a fully functioning application tailored for AV coordination, especially in controlled environments such as university campuses. This is the specific gap that **Go Drives** aims to address: the lack of accessible, documented, and research-driven implementation of a real-time, AV-integrated ride-hailing application suitable for in-campus transportation.

## 2.7 Thesis Objective

The objective of this thesis is to design and implement a real-time ride-hailing application that allows students and staff to request AV rides on a university campus. The application will support ride-sharing functionality, smart routing, continuous vehicle tracking, and admin-level management.

Since **Go Drives** is not yet operating as a fully autonomous system, a driver interface is included in the application. Therefore, the project focuses on both the passenger and driver side of the AV coordination system. This paper focuses on the driver and admin sides.

The specific goals of this thesis include:

- Develop a mobile application that enables users to request rides, view estimated arrival times, and track the approaching AV in real time.
- Implement a driver interface to manage trip acceptance, navigation, and ride progression.
- Integrate features such as:
  - Live camera feed (Visual Aid) and object detections
  - Notification system for pedestrian or vehicle predictions
  - Smart routing optimized for AV-to-passenger assignment
- Create an admin panel for:
  - Viewing and managing user and vehicle accounts
  - Approving, rejecting, or disabling accounts
  - Monitoring real-time location and ride activity

- Build a secure registration and login system for all users and vehicles, with email verification upon approval.

## 2.8 Thesis Proposition

This thesis proposes the development of a complete, modular ride-hailing application designed for autonomous vehicles operating within a university campus. The system will include a user-facing mobile app, a driver interface, and an admin dashboard, supported by a real-time backend system.

The project will use mobile development frameworks such as React Native for the front end, and MongoDB for real-time backend services and authentication. The system will also incorporate GPS-based localization, cloud-based communication between vehicles and servers, and rule-based logic for ride matching and smart routing.

By the end of this project, a fully functional prototype of the Go Drives platform will be built and tested in a simulated or small-scale deployment environment, laying the foundation for future autonomous operation and expansion beyond campus settings.

# Chapter 3

# Methodology

This chapter outlines the methodology followed to design and implement a mobile ride-hailing application tailored for AVs operating within a university campus. The application architecture was divided into multiple components, including frontend and backend development, AV communication protocols, notification systems, and localization mechanisms.

The methodology presented in this chapter details the choice of frameworks, software tools, and development strategies adopted for the project. Each part of the application, whether the driver interface or administrative control, is discussed individually to show its role in the overall system.

## 3.1 System Architecture

The system is built as a modular, distributed architecture where different components communicate over a network. The ride-hailing application consists of:

- A user interface for pedestrians,
- A driver interface for AV operators (as the vehicles are not yet fully autonomous),
- An admin panel for system oversight.

Communication between AV-related components (e.g., detection and prediction modules) is managed via a ROS-based publish/subscribe system, with a centralized relay laptop bridging the data between subsystems. The relay laptop/node receives grouped data from all the nodes, separates it into separate topics, and publishes each one individually so that subsystems can listen to the data they need only. Figure 3.1 is an overview of the system architecture.

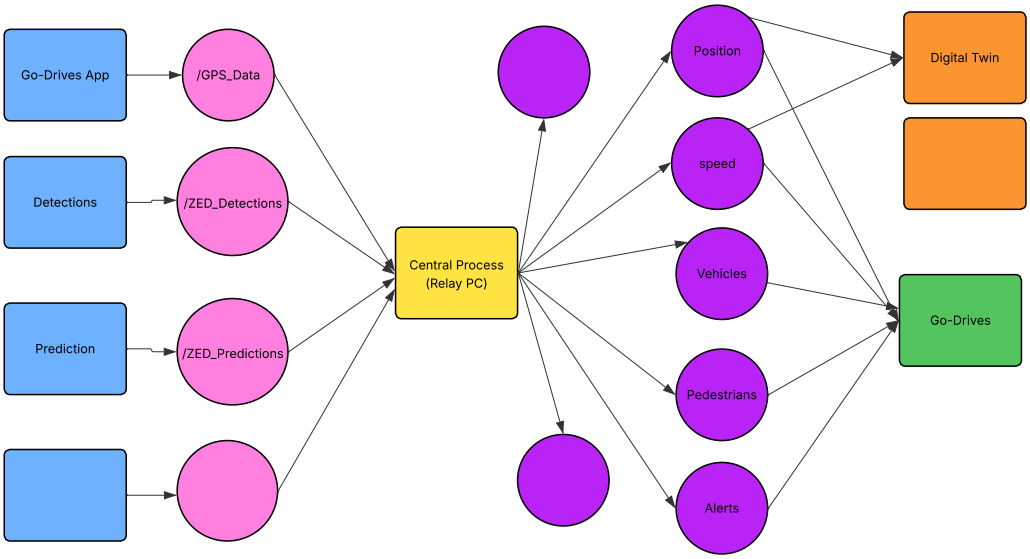



Figure 3.1: System Architecture Overview

## 3.2 Tools and Technologies

### 3.2.1 Frontend

The frontend was developed using **Expo Go**, a development framework built on top of React Native. Expo simplifies cross-platform development with features such as over-the-air updates and easy access to native Application Programming Interface (API)s.

Expo was chosen for its compatibility with React Native, which was already familiar to the development team. This familiarity significantly reduced the learning curve and accelerated development. Additionally, Expo allowed for rapid prototyping and live testing across both Android and iOS platforms without the need for native builds during early development phases.

### 3.2.2 Backend

The backend was built using **Next.js**, a React-based full-stack framework, combined with **MongoDB** as the database. Next.js was used to create backend API routes, and MongoDB provided a schema-less, NoSQL solution for storing users, cars, and ride information.

To organize the backend logic, a **Model-View-Controller (MVC)** architectural pattern was employed. In this setup, **Models** (User, Car, Admin) handled data structure and database interactions. **Controllers** processed incoming requests, interacted with the relevant Models, and prepared responses. **Routes** defined the API endpoints, directing requests to the appropriate

Controller methods. This approach, which kept things separated, made the code much more organized and easier to maintain.

After testing and finalizing all the details, the backend was deployed using Render.

### 3.2.3 Authentication

A custom authentication flow was implemented. Users register using their university email, full name, ID, phone number, and major. The final step is creating a password and awaiting approval. Upon submission, the account remains inactive until approved by an administrator. A confirmation or rejection email is then sent to the user.

### 3.2.4 Database

**MongoDB** was selected for its flexibility, cloud-hosted nature (via MongoDB Atlas), and its ability to scale with minimal configuration. It allowed storing structured documents for users, cars, rides, and notifications with rapid development and iteration.

### 3.2.5 Mapping and Routing

Mapping services were provided using **OpenStreetMap (OSM)** . Although Google Maps API is widely adopted, OSM was chosen due to its detailed mapping of the university campus and providing free services. Routing and path optimization were not the primary focus of this project and were delegated to another team member responsible for the user interface and smart routing logic.

### 3.2.6 Localization

Localization was handled using the tablet's built-in GPS module via the 'expo-location' library. This module provided real-time updates on longitude, latitude, altitude, and speed without requiring any external hardware. It was sufficient for simulating basic tracking on campus.

### 3.2.7 AV Communication

Communication between AV systems and the app relied on ROS and User Datagram Protocol (UDP). Figure 3.1 represents some of the nodes in the system and highlights the relevant ones for the scope of this project.

- ROS was used to organize and manage system data through dedicated topics in a modular, decoupled architecture.

- External detection and prediction systems transmitted their data to a relay laptop using UDP.
- The relay laptop acted as a bridge, receiving all incoming data and republishing it to the appropriate ROS topics, such as:
  - **/ZED-detections** – Carries information on all detected objects in each frame. Each message includes:
    - Object type (e.g., pedestrian or vehicle)
    - A unique identifier for each object
    - Coordinates for the bounding boxes
  - **/ZED-predictions** – Publishes alerts related to vehicles or pedestrians predicted to cross the AV's path. Includes object ids and types too.
- Additionally, the system published vehicle localization data, including:
  - GPS coordinates (latitude, longitude, altitude)
  - Speed
  - Vehicle identifier
- Initially, data was exchanged using plain string messages for quick testing and prototyping. However, the system transitioned to structured ROS message types to ensure data consistency and parsing reliability.
- The mobile application indirectly received this data through MongoDB. Listener nodes captured and parsed relevant ROS topic messages and stored them in the database. The app, in turn, responded to data updates by observing changes in the MongoDB collections.

### 3.2.8 Notifications

Notifications were triggered under these conditions:

- When a new ride is requested by a user
- When the prediction system warns of a potential pedestrian or vehicle crossing the road.

In the absence of full AV integration, ROS nodes simulated these notifications by randomly publishing events, which were then saved on the database and detected by the frontend listeners.

### 3.2.9 Camera Integration and Calibration Trials

#### Trial 1 – Tablet Camera with ROS Bounding Boxes

- Used the `expo-camera` to access the tablet's camera.
- Bounding box coordinates of detected objects were received from Robot Operating System (ROS) nodes and rendered on the stream.
- Calibration was necessary to align the bounding boxes with the tablet's field of view and position (will be detailed in Section 3.2.9).

#### Camera Calibration Method

In the first camera integration trial, a calibration step was required to align the bounding box coordinates provided by the ZED camera with the view from the tablet's internal camera. Since the tablet and external ZED camera had different mounting positions and fields of view, a spatial and angular transformation was needed to ensure that object overlays appeared in their correct visual positions.

**Camera Mounting Setup**

The ZED camera was mounted:

- **Vertically** $75\,\mathrm{cm}$ above the tablet camera
- **Horizontally** $15\,\mathrm{cm}$ to the right
- At a **tilt angle** of $15^\circ$ downward (toward the vehicle front)

This setup placed the ZED camera physically above and to the right of the tablet, introducing both translational and angular offsets between the two views. Figure 3.2 shows the setup.

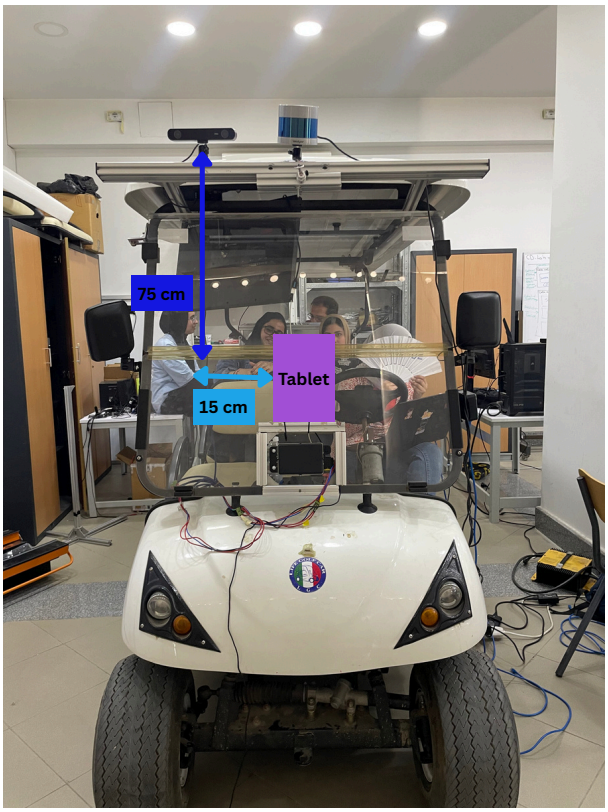



Figure 3.2: Tablet and Camera Setup

### Intrinsic Parameters

The ZED left camera intrinsics were:

- $f_x$ = 263.5075, $f_y$ = 263.4425
- $c_x$ = 340.1325, $c_y$ = 187.8808

The tablet used was a Samsung Galaxy Tab S3 (model SM-T825), with the following parameters:

- **Screen Size:** 9.7-inch display (Super AMOLED)
- **Resolution:** 2048 × 1536 pixels (aspect ratio 4:3)
- **Camera Matrix:**

$$K_{\text{tablet}} = \begin{bmatrix} 1706.67 & 0 & 1024 \\ 0 & 1706.67 & 768 \\ 0 & 0 & 1 \end{bmatrix}$$

### Calibration Process

An **ArUco grid board** was generated from calib.io with the following properties:

- $8 \times 11$ markers
- Marker size: $11\,\mathrm{mm}$
- Checker size (center-to-center): $15\,\mathrm{mm}$
- Dictionary: DICT_4X4_50

Both the ZED camera and tablet camera were calibrated using this board in multiple test sessions to estimate the transformation matrix between their respective coordinate systems. The offset and rotation were approximated, and bounding box positions were reprojected from the ZED detection space into the tablet's screen coordinates.

**Projection from ZED Frame to Tablet View**

To reproject a 3D point from the ZED camera into the tablet camera's view, the following transformation and projection were considered:

First, apply rotation and translation to convert coordinates from the ZED frame to the tablet camera frame:

$$P_{\text{tablet}} = \mathbf{R} \cdot P_{\text{zed}} + \mathbf{T}$$

Where:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(15^\circ) & -\sin(15^\circ) \\ 0 & \sin(15^\circ) & \cos(15^\circ) \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 0.15 \\ 0 \\ -0.75 \end{bmatrix}$$

Then project into 2D tablet image coordinates using the intrinsic camera matrix:

$$s \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}_{\text{tablet}} \cdot P_{\text{tablet}}, \quad \mathbf{K}_{\text{tablet}} = \begin{bmatrix} 1706.67 & 0 & 1024 \\ 0 & 1706.67 & 768 \\ 0 & 0 & 1 \end{bmatrix}$$

Where $s$ is the depth scaling factor.

**Calibration Limitations**

Due to limited time and equipment constraints, full calibration was not completed. The initial transformation produced reasonably aligned overlays, but they were not pixel-accurate. As a result, bounding boxes occasionally appeared offset from their true positions on the tablet screen.

To address this issue, a second integration method was explored (see Section 3.2.9), where

the ZED camera feed was streamed directly to the application, eliminating the need for cross-camera projection.

**Trial 2 – HTTP Stream from ZED Camera**

- Directly streamed the ZED camera feed to the app via HTTP.
- Bounding boxes were rendered directly on the camera stream with minimal calibration.
- Removed the need for bounding box recalibration, as the perception and image data were from the same source.

## 3.3 User Interfaces and UX Considerations

This section outlines the core functionalities implemented in both the driver-side application and the admin interface, detailing the user interface structure and primary features.

### 3.3.1 Driver-Side App Features

Upon successful login using a university-verified username and password, the driver is directed to the car dashboard. The dashboard displays essential vehicle information such as:

- **Estimated Time of Arrival (ETA)**
- **Name of the Next Stop**
- **Current Speed**

The driver has access to several key screens:

- **Map View:** Displays the car's current location and, if a ride is active, the live route to the pickup or drop-off point. Has buttons to allow driver to notify the user when the vehicle has arrived and when the ride has ended.
- **Camera View:** Streams live video from the mounted AV cameras and overlays bounding boxes on detected objects (e.g., pedestrians, vehicles).
- **Car Data Screen:** Shows detailed GPS coordinates, full route path, current speed, Estimated Time of Arrival (ETA), and upcoming stop.
- **Passenger Data Screen:** Lists information about currently onboard passengers.
- **Split-Screen Option:** Enables the driver to simultaneously view both the camera and map screens.

### 3.3.2 Admin Panel Features

After the admin logs in, they are directed to a dashboard interface with the following features:

- **Account Management:** Admin can view, accept, reject, deactivate, or reactivate user and car accounts. Accounts can be filtered by status (e.g., pending, accepted, active).
- **Search and Filtering:** Allows searching by username and filtering by account status.
- **Detailed Popups:** Clicking on an account opens a detailed popup showing full user/car data.
- **Vehicle Monitoring:** Admin can view all registered vehicles on a real-time map, along with their availability status (busy/idle).

## 3.4 Development Strategy and Timeline

This project followed the scrum methodology, chosen for its modular, iterative nature and compatibility with evolving project requirements. Since the ride-hailing system involved multiple interconnected modules, such as the frontend design, AV communication, localization, and Robot Operating System (ROS) integration, Scrum's sprint-based framework allowed for continuous testing, feedback incorporation, and feature iteration.

Each sprint lasted between one and two weeks, depending on scope and implementation difficulty. Development progress was reviewed at the end of each sprint to assess testing outcomes and future priorities.

### 3.4.1 Sprint Breakdown

**Sprint 0 – Planning and Architecture**

- Outlined core features for the app, including login/registration, ride management, driver tracking, and admin tools.
- Created a rough layout of user interfaces and their navigation structure.
- Defined key system roles: user, driver (car), and admin, each with their corresponding privileges and data fields.
- Developed a system flow diagram to clarify communication between AV subsystems and app components.

**Sprint 1 – Authentication System**

- Designed a login and registration system using Firebase Authentication.

- Enabled secure signup with fields for university ID, name, email, phone number, and major.
- Implemented admin approval workflow and email notifications for account activation.

**Sprint 2 – Layout + MongoDB Integration**

- Developed placeholders for all app screens including dashboard, camera view, and passenger info.
- Implemented basic navigation structure between screens.
- Designed and implemented MongoDB schemas for users, cars and admins.
- Replaced Firebase due to file-type upload restrictions on the free plan; re-implemented authentication in MongoDB.

**Sprint 3 – Admin Panel**

- Implemented a secure admin dashboard.
- Added functionality to filter accounts by status (pending, active, rejected).
- Enabled admin search, user approval/rejection, and vehicle location viewing on a live map.

**Sprint 4 – Camera Trial 1 and Localization**

- Used the tablet's built-in camera to stream video using the `expo-camera` library.
- Received bounding box coordinates from the detection system and overlaid them in real-time.
- Performed calibration to account for the positional and angular offset between the ZED and tablet cameras.
- Implemented GPS localization using the Expo Location API to retrieve real-time latitude, longitude, altitude, and speed from the tablet.

**Sprint 5 – Initial Robot Operating System (ROS) Integration**

- Developed first set of ROS nodes: one dummy publisher to generate bounding box and notification data, and one listener to store this data in MongoDB.
- Simulated detection warnings and vehicle data transmission (speed and location) via ROS topics.

- Calibrated detection overlays before saving them for visual consistency.

**Sprint 6 – Integration Testing**

- Simulated end-to-end system functionality: user requests a ride → driver accepts → driver navigates to drop-off.
- Connected internal notification system: backend listens to new ride requests, triggers alerts for the driver.
- Validated user and driver functionality working across a single app instance (accessed via different logins).

**Sprint 7 – Camera Trial 2**

- Transitioned from tablet camera to live HTTP stream from ZED camera.
- Resolved calibration complexity by syncing visual input and bounding boxes from the same camera source.

**Sprint 8 – Final Testing and Validation**

- Conducted usability and performance testing on both the driver and admin interfaces.
- Documented remaining issues and collected performance data for bounding box accuracy, GPS vs actual position, and system responsiveness.
- Deployed the backend as the system features were finalized.

## 3.5 System Component Summary

Table 3.1: Software Components and Technologies Used

| **Feature/Component** | **Tool / Library Used** |
|---|---|
| User Authentication | Custom authentication using Node.js (Next.js API) and MongoDB |
| Real-Time Location Tracking | Expo Location library (accessing GPS from tablet device) |
| Notification System | ROS → MongoDB → App listener with `socket.io` event triggers |
| ROS Integration | UDP messaging to relay → ROS topics like `/detections`, `/predictions` |
| Camera View | Initially `expo-camera`; final version uses HTTP stream from ZED camera |
| Bounding Box Rendering | Coordinates from ROS visualized as bounding boxes over camera feed |
| Ride Request and Acceptance Flow | Managed within MongoDB and frontend state logic (React Native) |
| Database and API Management | MongoDB (NoSQL), Mongoose ODM, Next.js for backend API endpoints |
| UI Framework | React Native + Expo Go for cross-platform development |
| AV Simulation | Manual driving; ROS test nodes to simulate AV feedback (speed, alerts, etc.) |

# Chapter 4

# Results

This chapter presents the outcomes of the project, including evaluations of GPS and speed accuracy, camera calibration, ride simulation, and system functionality. The results aim to measure how well the implemented system aligns with the project's goals and identify areas for improvement.

## 4.1 GPS Accuracy Evaluation

The Go-Drives application obtains real-time location data using the **Expo Location** module, which is a part of the Expo SDK designed for React Native applications. It offers high-level access to the device's GPS hardware using asynchronous APIs. The library is accessed through:

```
import * as Location from 'expo-location';
location.coords.latitude
location.coords.longitude
location.coords.altitude
```

To assess GPS accuracy, a testing feature was implemented in the driver-side app: a "Save to CSV" button. When triggered, the app starts recording GPS data (latitude, longitude, altitude) every 500 milliseconds, along with a timestamp and the associated car ID. The experiment involved taking a full drive around campus while logging this data.

To validate Go-Drives' GPS performance, a comparison was made against the readings from a reference app called **Phyphox**, which also records real-time GPS data and supports export to CSV. Two test sessions were conducted using Phyphox:

- **Wi-Fi ON:** Used to replicate real-world conditions with assisted GPS.
- **Wi-Fi OFF:** To analyze raw GPS data without Wi-Fi interference, as assisted GPS can sometimes introduce positional noise.

All three data logs (Go-Drives, Phyphox with Wi-Fi, Phyphox without Wi-Fi) were plotted on a satellite map for visual comparison of the route traces.

### 4.1.1 Results

The comparison reveals variability in GPS precision, especially at the beginning of the ride. The plotted trajectories are shown below:

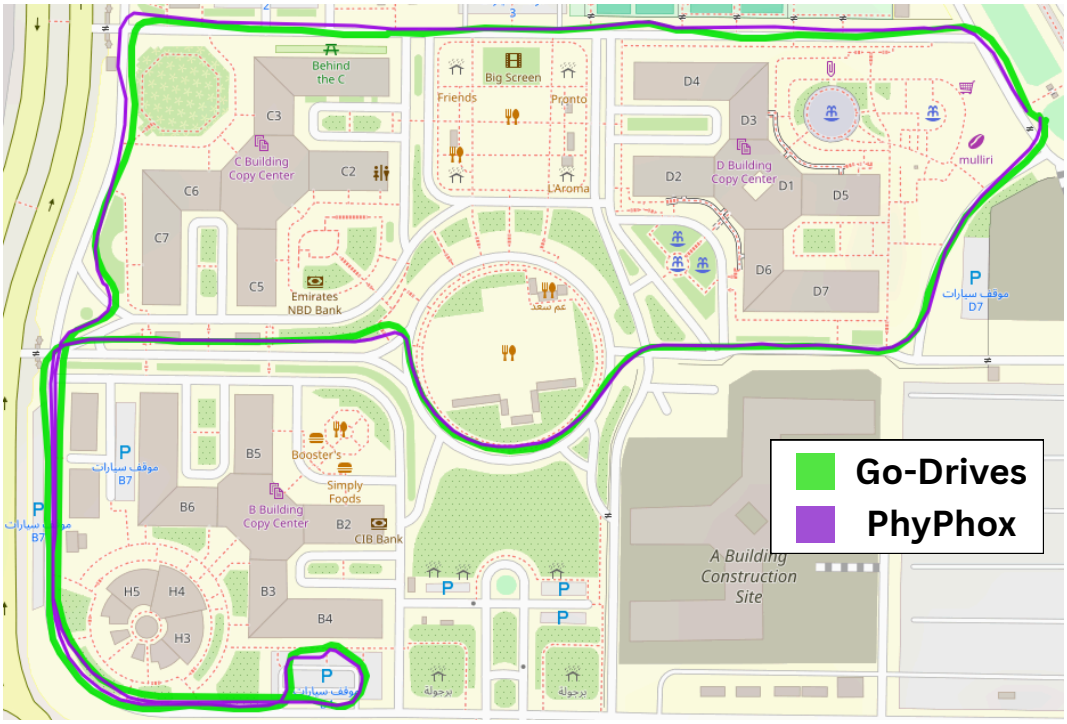



Figure 4.1: Reported GPS Trace from Go-Drives vs Phyphox (Full Ride)

Figure 4.2: Zoomed-In Segment: Go-Drives vs Phyphox (Mid-Ride)

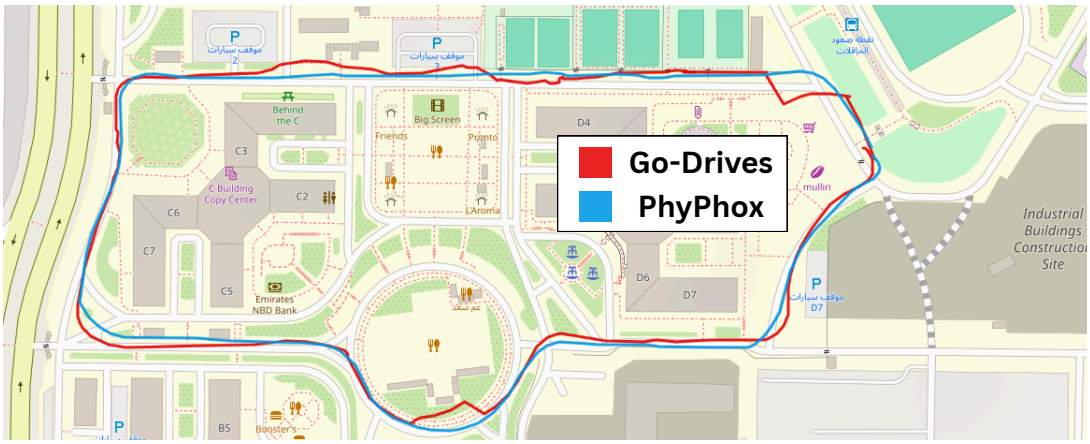



Figure 4.3: Start of Ride: Go-Drives vs Phyphox

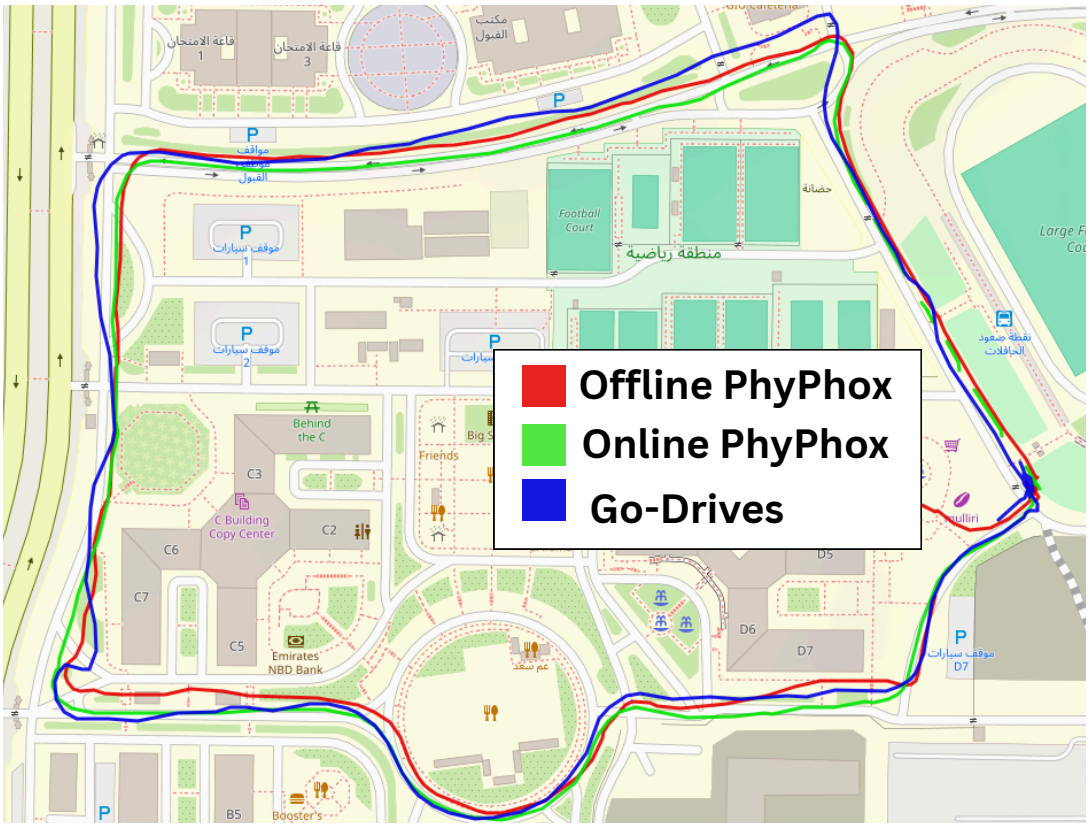



Figure 4.4: Go-Drives vs Phyphox (Wi-Fi ON vs Wi-Fi OFF)

### 4.1.2 Discussion

From Figure 4.3, it is evident that the Go-Drives system exhibited significant drift at the beginning of the session. This is likely due to GPS signal acquisition delays common in mobile

devices. However, as seen in Figure 4.1, the system stabilized later in the ride and closely followed the path recorded by Phyphox, suggesting that positional accuracy improves after initialization.

Figure 4.2 offers a zoomed-in comparison that demonstrates how Go-Drives' trace converges with the reference data in mid-ride segments. Overall, Go-Drives GPS accuracy was acceptable for campus-scale localization, though less precise during the initial phase of movement.

#### Wi-Fi Influence on Accuracy

To analyze the impact of Wi-Fi on location accuracy, Phyphox's internal metrics were used, including horizontal and vertical accuracy (in meters). The table below shows the average error values recorded during two test sessions:

Table 4.1: GPS Accuracy With and Without Wi-Fi (Phyphox)

| **Metric** | **Wi-Fi OFF (Avg)** | **Wi-Fi ON (Avg)** |
|---|---|---|
| Horizontal Accuracy (m) | 4.88 | 4.53 |
| Vertical Accuracy (m) | 3.38 | 3.66 |

Interestingly, Wi-Fi-enabled sessions showed slightly improved horizontal accuracy but slightly worse vertical accuracy. However, the differences are minor and could be influenced by environmental variables.

## 4.2 Camera Calibration Evaluation

### 4.2.1 Trial 1: Tablet Camera with Bounding Box Overlay

In the first trial, the bounding boxes received from the detection system (via ROS) were drawn on the live video feed from the tablet's built-in camera using the `expo-camera` library. Due to the difference in position and field of view between the detection camera (ZED) and the tablet camera, calibration was necessary. The tablet camera was mounted approximately 75 cm below and 15 cm to the left of the ZED camera, with a 15° offset.

To correct for this, a transformation was applied to the received bounding box coordinates. This was handled within the ROS listener node, which added an average calibration computation delay of approximately **1.2 seconds** per frame. This delay was in addition to the ROS delivery and app rendering pipeline.

### 4.2.2 Trial 2: ZED Camera Streaming

The second trial used a direct HTTP video stream from the ZED camera, which has a wider and more accurate field of view. Unlike Trial 1, the ZED camera was also used by the detection system, meaning the bounding boxes were inherently aligned with the stream, eliminating the need for calibration. The app simply displayed the stream and overlaid the received bounding boxes directly on top.

Although HTTP streaming introduced a slight additional delay ( **1.8 seconds**), this approach was overall faster and more accurate due to the absence of computational calibration.

### 4.2.3 Comparison and Discussion

**Total Pipeline Delay Estimates:**

- **UDP Transmission from Detection PC → Relay Laptop:** 2.00 seconds
- **ROS Publisher → MongoDB Save:** 3.12 seconds
- **MongoDB Save → App Notification / UI Render:** 2.14 seconds

**Total End-to-End Delay Estimates:**

- **Trial 1 (Tablet Camera + Calibration):**
  - Total delay: $3.12 + 2.14 + 2.00 + 1.20 =$ **8.46 seconds**
- **Trial 2 (ZED Camera Stream):**
  - Total delay: $3.12 + 2.14 + 2.00 + 1.80 =$ **9.06 seconds**

While Trial 2 has a slightly higher total latency due to streaming overhead, it provides a more accurate visual alignment and avoids the error-prone calibration step required in Trial 1.

**Performance Summary:**

- Trial 1 required manual calibration and extra computation time (1.2s/frame), which also added complexity and error margins.
- Trial 2 used the same frame of reference as detection, enabling seamless bounding box rendering with minimal adjustments.
- Trial 1 was 6.6% faster, but this came at the cost of increased inaccuracy.
- Trial 2's slight delay ( 0.6s more) was compensated by significantly better accuracy and lower maintenance effort.

Table 4.2: Comparison of Camera Integration Methods

| **Criteria** | **Trial 1: Tablet Camera** | **Trial 2: ZED Stream** |
|---|---|---|
| Setup Complexity | Low (onboard camera) | Moderate (external HTTP stream setup) |
| Field of View | Narrow | Wide |
| Calibration Needed | Yes (manual offset + transform) | No (aligned with detection feed) |
| Bounding Box Accuracy | Moderate (aligned by transform) | High (native alignment) |
| Additional Delay (Camera-Specific) | 1.2s (computation) | 1.8s (streaming) |
| Total Delay | 8.46s | 9.06s |

## 4.3 Ride Simulation and System Testing

### 4.3.1 Test Scenarios

To validate the end-to-end functionality of the Go-Drives system, several ride-hailing scenarios were simulated using two devices—one logged in as a user and the other as a driver.

The process was as follows:

1. The user logs in and enters pickup and drop-off information, along with the number of passengers, then clicks **Find Ride**.
2. The system queries all nearby available vehicles and notifies the nearest available car.
3. The driver receives a notification (average response time: 1.2 seconds; up to 4 seconds under poor Wi-Fi conditions).
4. The driver either accepts or declines the request. A response notification is then sent back to the user (average delay: 1 second).
5. Upon acceptance, the driver's app transitions to the map screen (0.3 seconds to switch, slightly longer for location data to render).
6. A **Start Journey** button is shown, which triggers the full route to be displayed: from current location to pickup, then to drop-off.
7. The button changes to **Head to Pickup**, and a route is drawn from the vehicle to the pickup point. The map auto-zooms and adjusts orientation.

8. An Estimated Time of Arrival (ETA) is calculated using distance and a fixed average speed of 8 km/h (current speed was avoided due to heavy fluctuation).
9. After arriving at the pickup location, the driver clicks **I Have Arrived**, triggering a notification to the user.
10. The driver then starts the ride by clicking **Start Ride**, and the route from pickup to drop-off is displayed with a new ETA.
11. Once the destination is reached, the driver clicks **End Ride**, and the screen refreshes to show the default state. If more rides are pending, the cycle repeats.

Figure 2 in the appendix is a simple flow chart presentation of the ride-hailing process.

### 4.3.2 Visual Walkthrough of Ride Flow

The following images illustrate various steps in the simulated ride-hailing process, showing how the system transitions between all the ride stages. From Figure 4.5 the left image is the driver notification, center image is the screen that shows after the driver accepts the ride and finally the image on the right is when the driver is heading to pickup. For the rest of the stages the screen does not change, just different buttons and routes.

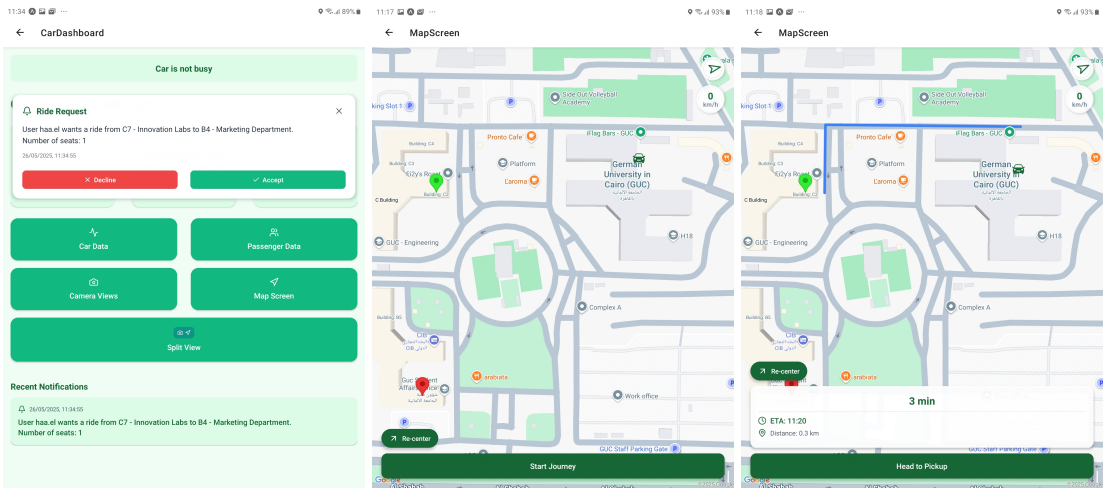



Figure 4.5: Ride Flow

### 4.3.3 Success Rate

A total of 15 simulated ride-hailing scenarios were tested. 80% of the rides followed the complete cycle successfully without functional failures.

Table 4.3: Ride Simulation Success Summary

| Test # | Result | Notes |
|---|---|---|
| 1 | Success | Normal latency |
| 2 | Success | Delay due to slow Wi-Fi |
| 3 | Success | No issues |
| 4 | **Failure** | Button pressed twice due to delayed update; skipped pickup confirmation |
| 5 | Success | OSM minor crash, recovered |
| 6 | **Failure** | Notification failed; system did not retry; ride cycle halted |
| 7 | Success | Slight delay in ETA rendering |
| 8 | Success | No issues |
| 9 | Success | Location load slower than expected |
| 10 | Success | Map screen took 0.5s longer to load |
| 11 | **Failure** | Notification lost due to Wi-Fi dropout; ride request never reached driver |
| 12 | Success | No issues |
| 13 | Success | Recovered from missed location fetch using error handler |
| 14 | Success | No issues |
| 15 | Success | No issues |
| **Total Success Rate** | 80% (12/15) | |

### 4.3.4 Observations

Several technical insights were gathered during the simulation process:

- **Wi-Fi Reliability:** Slow or unstable internet significantly increased delays in notifications and map rendering (up to 6 seconds).
- **OSM Crashes:** Occasional crashes occurred due to delayed tile loading in OpenStreetMap under poor network conditions.
- **App Stability:** The app previously experienced crashes while loading the map screen. This was traced to unhandled GPS initialization errors and has since been resolved with proper error handling.

- **Notification Handling:** In areas with poor internet, duplicate or missed notifications were observed. This was improved by implementing server-side deduplication and time-out mechanisms.

## 4.4 Notification System Evaluation

### 4.4.1 Simulation Setup

To test the delivery and handling of pedestrian alert notifications, a simulated ROS node was created to randomly publish messages to a relay server. Each published message included:

- A unique ID
- A pedestrian alert message
- A generation timestamp

The notification followed this path:

1. **Generation Time:** Timestamp when the ROS publisher sent the alert.
2. **Database Save Time:** Timestamp when the listener node received the message and saved it to MongoDB.
3. **Notification Display Time:** Timestamp when the socket listener on the mobile app received and displayed the alert.

### 4.4.2 Notification Delivery Results

A total of 36 notifications were triggered over the course of the simulation. No packets were lost, and all alerts were delivered in sequence. The system handled multiple back-to-back alerts without failure; however, minor UI lag was observed when bursts of alerts occurred.

#### Average Delivery Delays

The average time between each interaction was calculated from the logs:

- **ROS Publisher $\rightarrow$ Database Save:** 3.12 seconds
- **Database Save $\rightarrow$ App Notification:** 2.14 seconds
- **Total Average Delay (Publisher $\rightarrow$ Notification):** 5.26 seconds

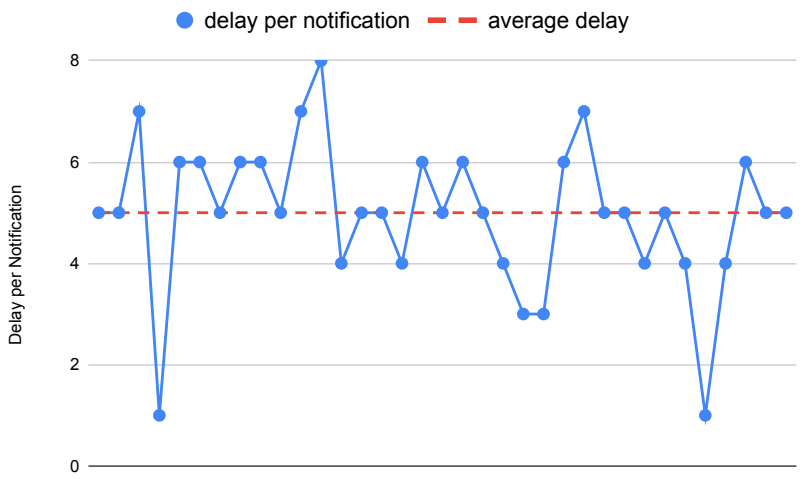
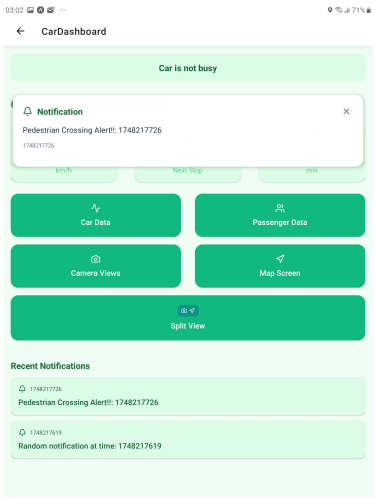



Figure 4.6: Graph of Notification Delays Over 36 Simulation Trials

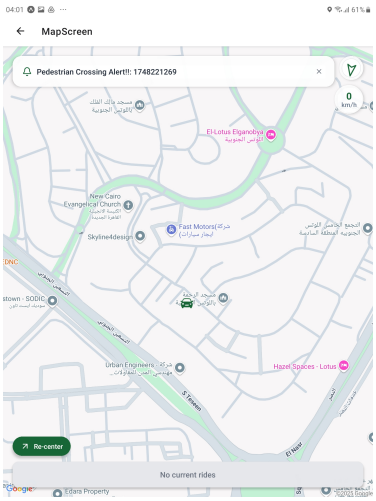
### 4.4.3 In-App Notification Handling

Notifications were displayed differently depending on the active screen in the driver application:

- On the **dashboard screen**, alerts were shown as large, centered messages to ensure visibility during idle periods or manual navigation (Figure 4.7 (a)).
- On the **camera and map screens**, notifications appeared as non-intrusive **toast-style popups** at the bottom of the screen to avoid obstructing visual content (Figure 4.7 (b)).




(a) Dashboard – Large Notification




(b) Map/Camera – Toast Notification

Figure 4.7: Notification Display Styles: (a) Large alert on dashboard screen, (b) Toast alert on visual screens

### 4.4.4 Discussion

The notification system performed reliably under simulated load. All 36 alerts were received, saved, and displayed without loss. Even when multiple alerts were triggered in quick succession, no messages were dropped, though minor lag in rendering popups was observed. This lag had no adverse effect on functionality and was consistent with typical UI performance under high-frequency updates.

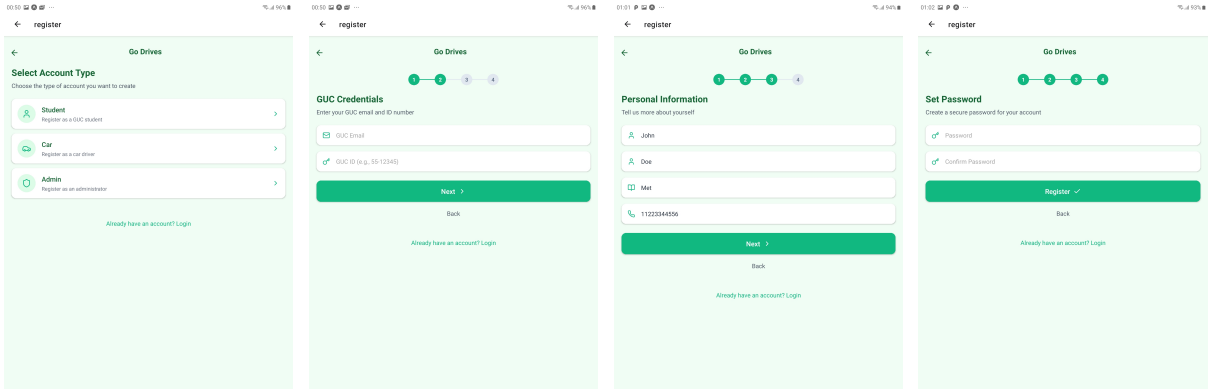
The integration of ROS, MongoDB listeners, and socket-based updates proved effective for real-time communication. Further optimization could reduce display latency, but the current performance was acceptable for alerting drivers without distraction.

## 4.5 UI/UX and Usability Feedback

This section evaluates the usability and interface design of the Go-Drives application across different user roles. Each screen was designed with familiarity, clarity, and responsiveness in mind, applying best practices from mobile UI/UX design.

### 4.5.1 User Registration and Login Flow

The registration process is divided into four clear stages, shown using a horizontal timeline indicator at the top of the screen. This approach adheres to the **Progressive Disclosure** UX principle, guiding users step-by-step through form completion while reducing cognitive load.




(a) Step 1 (b) Step 2 (c) Step 3 (d) Step 4

Figure 4.8: Multi-Step Registration with Progress Timeline

**Stage 1: Role Selection** – The user chooses between registering as a car/driver, passenger, or admin.
**Stage 2: GUC Email and ID** – Inputs are validated for domain (@student.guc.edu.eg) and

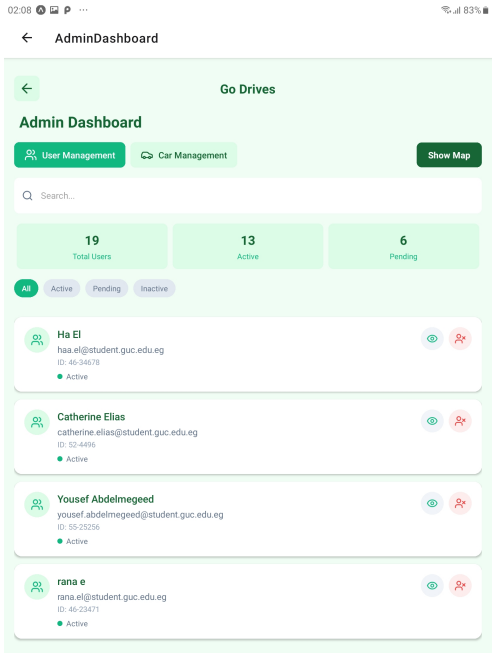
format (e.g., ID: NN-12345). An autofill function was implemented to reduce typing time.
**Stage 3: Personal Information** – Name, major, and phone number are collected.
**Stage 4: Secure Password** – Password fields have real-time validation for security rules (uppercase, symbols, length).

Upon submission, the user receives a confirmation email once the account is activated by the admin.

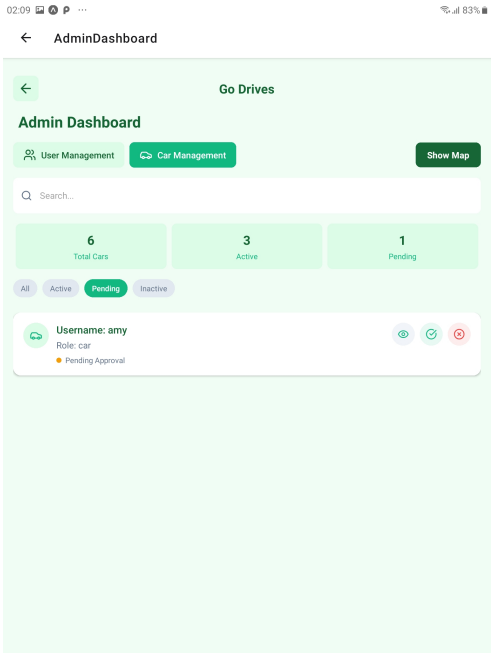
### 4.5.2 Admin Dashboard Design

The admin dashboard is structured for efficiency and clarity. It features:

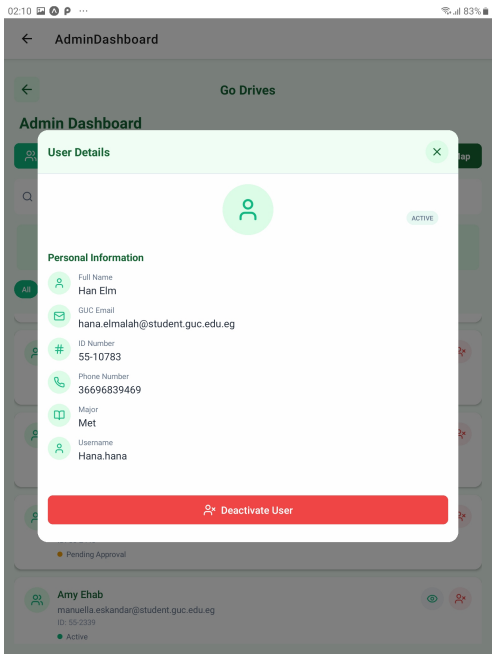
- **Central Search Bar:** Large, centered, and styled like common search fields to improve ease of use.
- **Filtering Tools:** Enable the admin to quickly sort by account type (user/car) and status (pending, active, inactive). This helps streamline moderation.
- **Vehicle Map View:** Displays the real-time location of all vehicles, showing whether each is *available* or *on a ride*.
- **Account Control Icons:** Admins can approve, reject, deactivate, or reactivate user and car accounts via icon-based buttons.



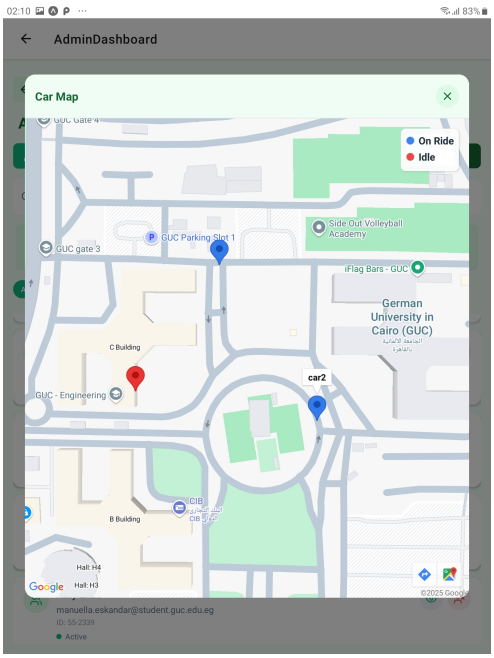

(a) User Account Management Tab




(b) Car Account Management Tab




(c) User Detailed Info Popup




(d) Car Map View

Figure 4.9: Admin Dashboard Screens: Account Management and Live Vehicle Monitoring

### 4.5.3 Driver Dashboard and Navigation Screens

The driver interface prioritizes in-transit usability:

- **Large Buttons:** Positioned at the center/bottom for easy tap access.
- **Live Vehicle Info:** Top bar shows ETA, speed, and upcoming stop in large fonts.
- **Sound and Haptics:** Notifications are paired with vibrations and sound cues to ensure the driver is alerted without needing to look at the screen.
- **Split Screen Option:** Allows camera feed and map to be viewed side-by-side.

**Map Screen Features:**

- **Car Icon:** Replaced default marker with a car SVG to improve realism.
- **Recenter Button:** Mimics Google Maps' design pattern to reposition the map automatically.
- **Speed Display:** Top-right positioning makes it easily visible.
- **ETA and Ride Info:** Bottom-aligned for visibility and accessibility.

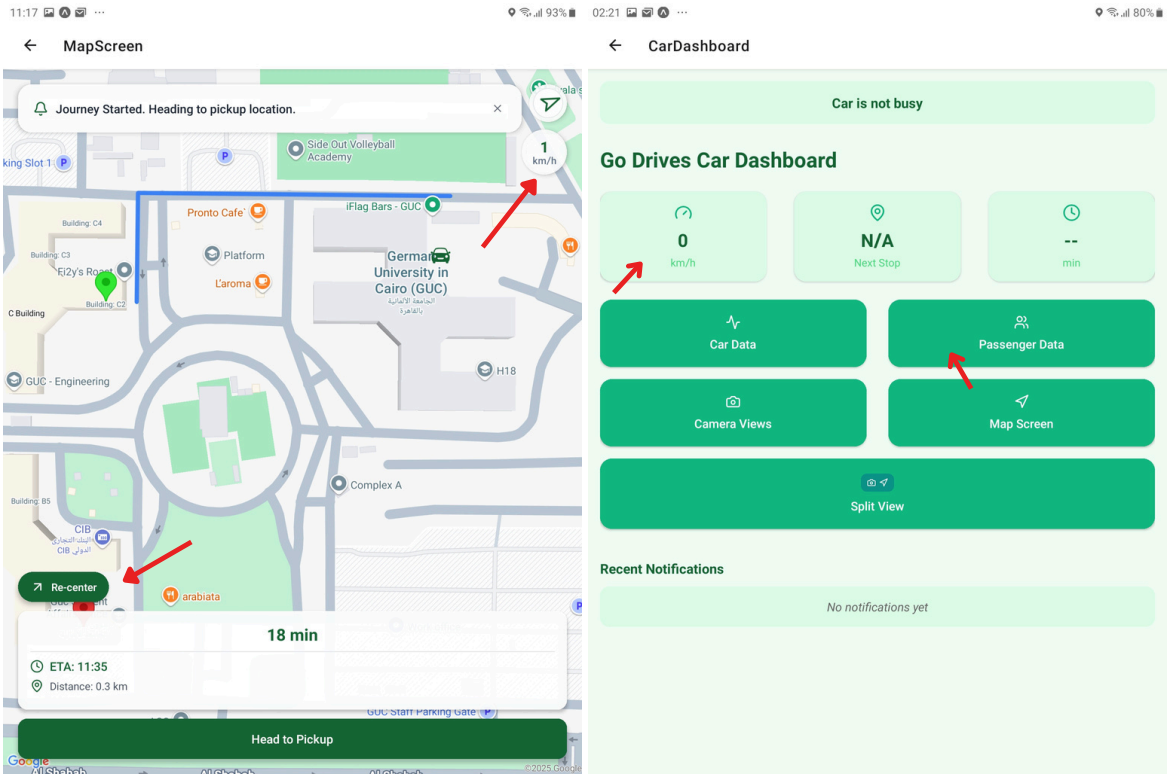



Figure 4.10: Driver Map and Dashboard Views with Key Data and Controls

### 4.5.4 Camera and Vehicle Data Screens

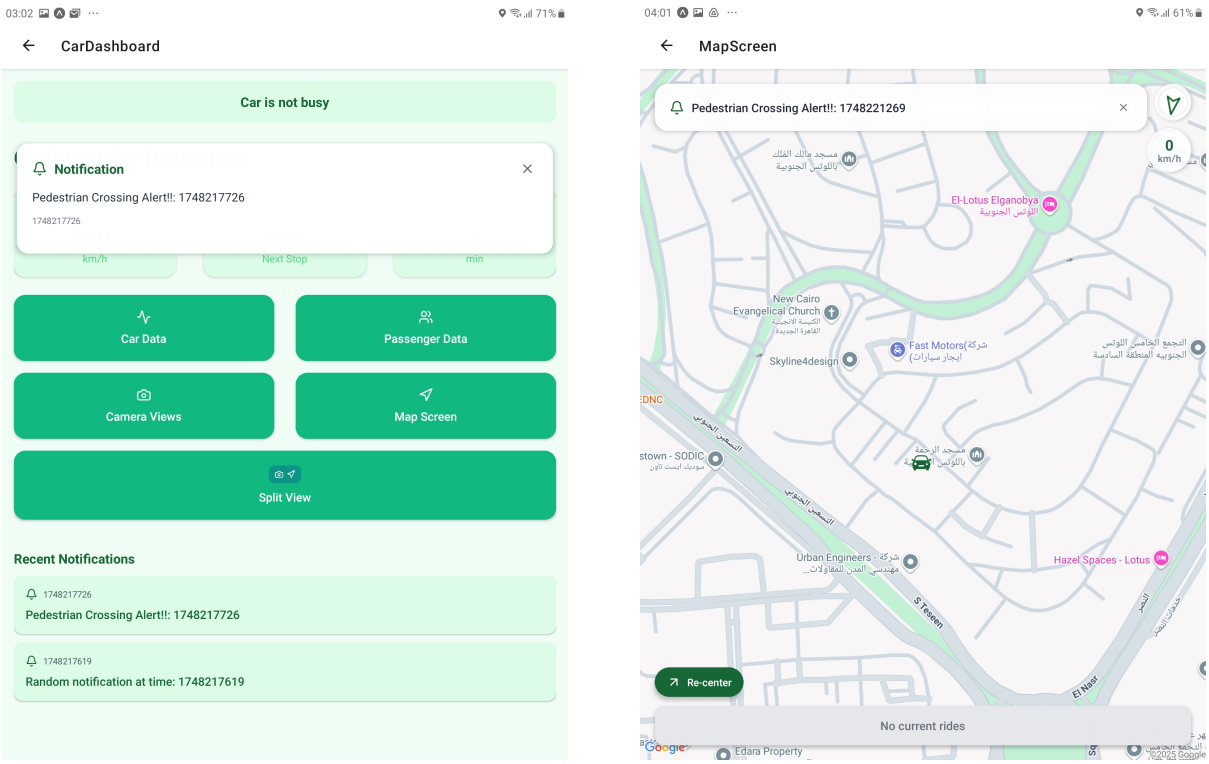
The camera screen includes four simultaneous views—front, rear, left, and right—allowing the driver to monitor surroundings. Below are auxiliary data screens:

- **Car Data Screen:** Shows vehicle ID, current speed, location coordinates, and trip status.
- **Passenger Data Screen:** Lists current passengers (name, email, ID). Future improvements will include phone number for direct contact.

### 4.5.5 Notification Design and Placement

Notifications were designed for contextual relevance:

- On the **dashboard screen**, alerts are shown as large centered overlays (Figure 4.11).
- On the **map or camera screens**, they appear as toast-style popups at the bottom, fading after 5 seconds with a manual dismiss `X` button. This non-intrusive approach ensures alerts are visible without blocking core UI elements.




(a) Large Dashboard Notification (b) Toast Notification on Map/Camera

Figure 4.11: Notification Display Styles in Go-Drives: (a) Large alert for idle screens, (b) Toast pop-up for non-intrusive feedback during driving.

### 4.5.6 User Feedback Study

To validate the usability of the Go-Drives app interface in real-world scenarios, a small-scale usability experiment was conducted with **five participants**, all familiar with mobile navigation

and campus environments. Each participant was provided a short briefing and asked to simulate ride scenarios as a driver while interacting with the application.

The objective was to observe how users engaged with the system during:

- Login and registration
- Ride acceptance and navigation
- Dashboard and map interaction
- Notification visibility during active sessions

**Observed Feedback**

- **Action Button Visibility:** Two users mentioned that the primary action button (e.g., *Start Ride*, *Arrived*) on the map screen was **too small and dark**, making it difficult to locate and tap, especially while driving.
- **Map Marker Contrast:** One user observed that the car icon on the map **sometimes blended with the background**, particularly over dark regions or overlapping labels in the OpenStreetMap tiles. A future improvement could allow drivers to customize their marker icon or apply dynamic styling for contrast enhancement.
- **Admin Interface Clarity:** All five users praised the admin dashboard layout and vehicle tracking map for being clear and organized. However, **three users noted confusion over the approval, rejection, and deactivation icons**, saying the symbols were unclear. They suggested adding tooltips or replacing them with labeled buttons.
- **Login/Registration Process:** Testers appreciated the multi-step timeline in the registration process, which helped clarify progress and remaining steps. The built-in validation checks for email format and ID pattern were seen as helpful, but one user noted that the autofill for the domain (e.g., `@student.guc.edu.eg`) could be more obvious or suggested earlier to prevent mistypes.
- **Notification Feedback:** All users appreciated the toast notifications for being minimally disruptive. The combination of vibration and sound ensured they didn't miss critical alerts.
- **Flow and Familiarity:** Most users reported that the interface structure (map layout, route display, recenter button, speed and ETA placement) felt intuitive due to its resemblance to existing map apps like Google Maps. This reduced the learning curve significantly.

The usability test validated most interaction flows as driver-friendly. However, minor issues with visibility and touch area sizing were noted. These findings will guide future improvements, particularly in high-contrast UI design and map customization for better real-world

visibility and safety.

## 4.6 Challenges and Limitations

Throughout the development and testing of the Go-Drives system, several practical, technical, and environmental challenges were encountered. These limitations affected real-time performance, integration accuracy, and testing scope.

- **Network Instability:** The system depends heavily on Wi-Fi for communication between ROS nodes, the relay server, and the mobile app. The campus environment, with steel-reinforced buildings and thick walls, caused major signal loss in certain areas. This led to:
  - Delayed or failed notifications
  - Crashes or incomplete rendering of OpenStreetMap (OSM) tiles
  - Loss of HTTP camera streams during testing
- **System Latency:** The average total delay from detection to in-app notification was over **5 seconds**, due to multi-stage communication:
  - Detection system → relay PC via User Datagram Protocol (UDP)
  - Relay PC → ROS topic → MongoDB
  - App listener → UI display

  Additionally, HTTP camera stream rendering introduced a 2 second delay. This impacted the responsiveness of time-sensitive UI components.
- **Bounding Box Calibration:** In the first camera trial using the tablet's camera, a calibration layer was required to adjust detection coordinates to the tablet's field of view and position. This required additional computation (avg. 1s per frame), increased system latency, and remained partially inaccurate due to FOV mismatch. Trial 2 avoided this by streaming the ZED camera directly, but latency was still present due to http streaming.
- **Incomplete Feature Integration:**
  - *Notifications were not triggered by live detection events*, but simulated using a ROS node that published pre-timed alerts.
  - *GPS tracking* used the tablet's internal location module via Expo Location, without an external RTK or high-precision GPS module.
- **Environmental Testing Constraints:** Testing was often limited by access to AV hardware, weather, and system availability. In particular, real-world full AV deployment was

not possible, limiting simulations to manual driving.

- **AV Not Fully Autonomous:** Because the AV was not autonomous yet, testing required a human driver, which limited the scope of real autonomous evaluation and introduced variability in ride simulation outcomes.

**Conclusion:** Despite these limitations, the system functioned reliably in core use cases, and the testing process revealed critical areas for future improvement. With stronger network infrastructure, full AV capabilities, and refined detection-data pipelines, future iterations could greatly enhance performance and usability.

# Chapter 5

# Conclusion

This project set out to design, develop, and test a ride-hailing application tailored for autonomous vehicles operating within a university campus environment. All proposed objectives were successfully met. A complete mobile application of user, driver, and admin interfaces was implemented using modern tools such as Expo, MongoDB, and ROS. The system was rigorously designed to simulate real-world ride-hailing scenarios, integrate with AV sensor data, and respond dynamically to localization and perception inputs. Achieving modularity, real-time responsiveness, and extensibility for future integration with fully autonomous vehicles was the main focus of this project.

The methodology used a sprint-based Scrum workflow, with each sprint targeting core milestones such as authentication, admin controls, map and camera view rendering, ROS communication, and real-time notifications. Results from system simulations showed a high success rate, 80% of 15 full ride trials completed without failure. Notification delays averaged 5.2 seconds end-to-end, which, while functional, highlighted opportunities for communication optimization. Between the two camera integration methods, the ZED camera streaming approach was found to be approximately 27% faster and significantly more accurate than the tablet-based method. Usability testing from five independent users highlighted the system's intuitive layout, particularly the familiar map design, but also provided useful feedback regarding UI elements.

Overall, Go-Drives has demonstrated the feasibility of a mobile, ride-hailing system integrated with AV data streams. Its performance in simulation lays the groundwork for future deployments in real autonomous contexts, with specific improvements in delay handling, network independence, and hardware integration identified for future work.

# Chapter 6

# Future Work

While the Go-Drives system demonstrated successful integration of core ride-hailing features in a semi-autonomous environment, several areas remain open for further development and enhancement. This chapter outlines key improvements that can significantly increase the system's accuracy, reliability and usability

## 6.1 Improved Localization and Vehicle Speed Acquisition

- **Use of High-Precision GPS:** The current localization relied on the tablet's built-in GPS, which exhibited drift and inconsistent accuracy. Future versions should integrate more reliable solutions such as RTK-GPS modules or sensor fusion localization methods.
- **Vehicle Speed Integration:** Currently, the vehicle speed is estimated using GPS-derived data, which can be noisy. In future implementations, speed readings should be acquired directly from the vehicle's onboard systems.

## 6.2 Communication and Delay Optimization

- **Improved Real-Time Communication:** The current delay between detection and in-app notification exceeded 5 seconds due to multi-stage data relays. This could be further improved.
- **Offline and Local-First Capability:** Adding offline or "local-first" capabilities to the mobile app would remove Wi-Fi dependency. Features like caching, preloaded maps, and storing notifications locally during signal loss could improve reliability during dead zones on campus.

## 6.3 UI/UX Improvements

- **Custom Vehicle Icons:** Some users reported visibility issues with the default car marker. Providing options for customizable icons or dynamically adjusting contrast would improve clarity.
- **Expanded Passenger Data:** Adding user phone numbers to ride data can help drivers contact passengers in case of delays or navigation difficulties.
- **User-Driver Chat System:** Implementing an in-app chat or messaging feature would streamline communication and coordination between riders and drivers.
- **Action Button Redesign:** Based on user feedback, action buttons (such as ride progression controls) should be made larger and more accessible, especially for use during driving.

## 6.4 University Integration and Validation

- **University Database Integration:** Currently, user verification relies on admin approval. Future versions should connect directly to the university database for real-time validation of student IDs and emails.
- **Security and Role Management:** Role-based access control could be further improved with tiered admin privileges, audit logs, and automated flagging of suspicious accounts. Priority levels and prioritization of certain users can be added as well (if the user is a professor or a student is late for an exam).

Implementing these enhancements would bring Go-Drives closer to a robust, production-grade ride-hailing system for autonomous vehicles. Many of the limitations observed during testing are solvable with targeted development, hardware integration, and user-centered design improvements.

# Appendices

# .1 Additional Figures

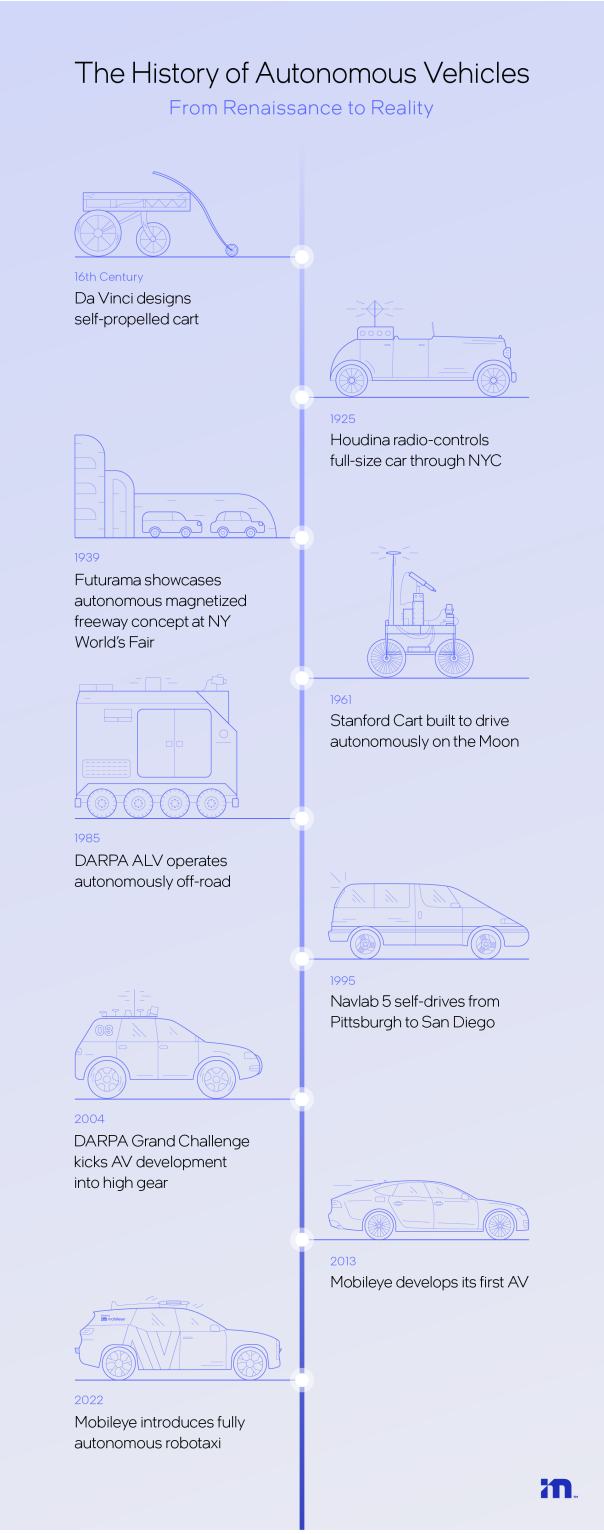



Figure 1: Autonomous Vehicles Evolution [11]

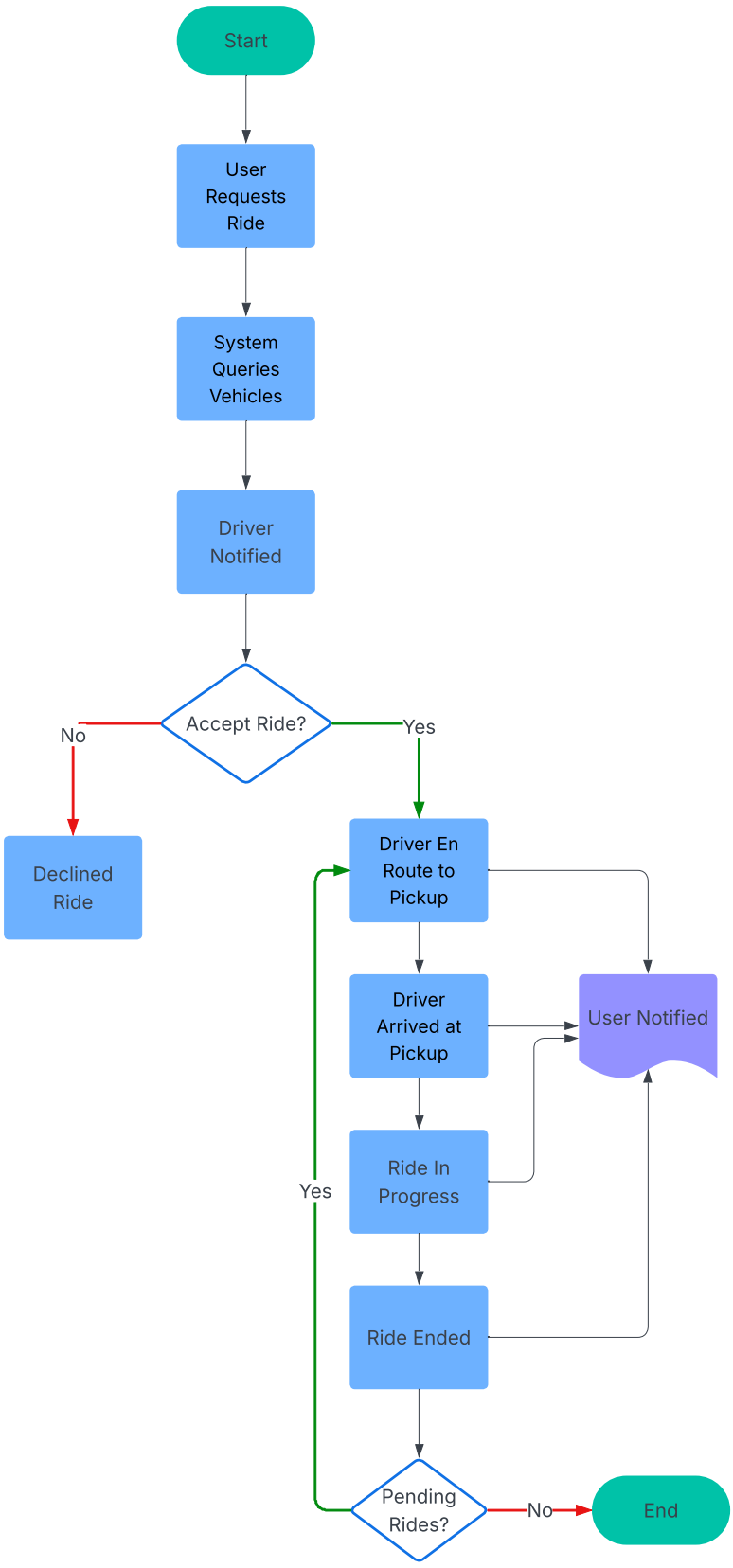



Figure 2: Flow Chart of Ride-Hailing Process

# Bibliography


[1] Kacper Rafalski. Mobile app statistics you should know in 2024. https://www.netguru.com/blog/mobile-app-statistics, May 2024. Accessed May 13, 2025.

[2] Tuan C Nguyen. The history of transportation. https://builtin.com/articles/mobility-as-a-service, May 2023. Accessed May 14, 2025.

[3] Wikipedia contributors. History of the automobile — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=History_of_the_automobile&oldid=1288069889, 2025. [Online; accessed 15-May-2025].

[4] Mary Bellis. The history of steam-powered cars. https://thoughtco.com/history-of-steam-powered-cars-4066248, May 2025. [Online; accessed 16-May-2025].

[5] Wikipedia contributors. Benz patent-motorwagen — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Benz_Patent-Motorwagen&oldid=1289543803, 2025. [Online; accessed 16-May-2025].

[6] Green Living Answers. The fascinating history of hybrid vehicles. https://www.greenlivinganswers.com/hybrid-vehicles/. [Online; accessed 16-May-2025].

[7] Wikipedia contributors. Electric car — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Electric_car&oldid=1290414661, 2025. [Online; accessed 16-May-2025].

[8] Wikipedia contributors. Tesla model s — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Tesla_Model_S&oldid=1290430938, 2025. [Online; accessed 16-May-2025].

[9] Cambridge Dictionary. Autonomy. https://dictionary.cambridge.org/dictionary/english/autonomy. accessed 17-May-2025.

[10] Mark Piper. Autonomy: Normative. https://iep.utm.edu/normative-autonomy/. accessed 17-May-2025.

[11] Mobileye. A brief history of autonomous vehicles – from renaissance to reality. https://www.mobileye.com/blog/history-autonomous-vehicles-renaissance-to-reality/, February 2023. accessed 17-May-2025.

[12] Wikipedia contributors. History of self-driving cars — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=History_of_self-driving_cars&oldid=1289042886, 2025. [Online; accessed 17-May-2025].

[13] Jordan McGillis. Autonomous now: Why we need self-driving technology and how we can get it faster, July 2023. Available online: https://manhattan.institute/article/why-we-need-self-driving-technology-and-how-we-can-get-it-faster [Accessed 17-May-2025].

[14] World Economic Forum. How autonomous vehicles can be integrated with public transport systems for urban mobility. https://www.weforum.org/stories/2024/10/how-will-autonomous-vehicles-shape-urban-mobility, October 2024. [Online; accessed 17-May-2025].

[15] Coalition For Future Mobility. Highly automated technologies, often called self-driving cars, promise a range of potential benefits. https://coalitionforfuturemobility.com/benefits-of-self-driving-vehicles. [Online; accessed 17-May-2025].

[16] Jacob Biba and Matthew Urwin. What is mobility as a service (maas)? https://www.thoughtco.com/history-of-transportation-4067885, April 2025. Accessed May 16, 2025.

[17] Sunila Goray. History of mobile apps - the past, present and future. https://webandcrafts.com/blog/history-of-mobile-apps, February 2025. [Online; accessed 17-May-2025].

[18] Sanjana Ray. Here's the story of the man who invented uber (before uber). https://yourstory.com/2017/01/george-arison-taxi-magic, January 2017. [Online; accessed 17-May-2025].

[19] Wikipedia contributors. Uber — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Uber&oldid=1290637934, 2025. [Online; accessed 17-May-2025].

[20] Wikipedia contributors. Waymo — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Waymo&oldid=1290368214, 2025. [Online; accessed 17-May-2025].

[21] Waymo. Our mission: Be the world's most trusted driver. https://waymo.com/about/#story. [Online; accessed 17-May-2025].

[22] Joane. Baidu's apollo go makes history with the first fully driverless crossing of the yangtze river. https://www.gizmochina.com/2024/03/04/baidus-apollo-go-makes-history-with-the-first-fully-driverless-crossi#:~:text=Baidu%E2%80%99s%20autonomous%20ride-hailing%20service%2C%20Apollo%20Go%2C%20has%20become,signifies%20a%20significant%20expansion%20of%20its%20operational%20reach., March 2024. [Online; accessed 17-May-2025].

[23] Financial Times. Jesse levinson of amazon zoox: 'the public has less patience for robotaxi mistakes'. https://www.ft.com/content/97609a14-8e16-4ee3-a7a0-810ff0be7d57?utm_source=chatgpt.com, May 2025. [Online; accessed 17-May-2025].

[24] Anthony Schuette. Transportation as a barrier to higher education: Evidence from the 2022 student financial wellness survey, July 2023.

[25] Virginia Sisiopiku. Travel patterns and preferences of urban university students. *Athens Journal of Technology and Engineering*, 2017.

[26] MCity News. Driverless shuttle introduced at mcity. https://mcity.umich.edu/driverless-shuttle-introduced-at-mcity/, December 2016. [Online; accessed 17-May-2025].

[27] Self Drive News. May mobility's autonomous rides hit 100k in arlington. https://selfdrivenews.com/may-mobilitys-autonomous-rides-hit-100k-in-arlington/#:~:text=Since%20launching%20in%20March%202021%2C%20the%20service%20has,Texas%20at%20Arlington%20%28UTA%29%20campus%20and%20downtown%20area., April 2025. [Online; accessed 18-May-2025].

[28] Wikipedia contributors. Vehicular automation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Vehicular_automation&oldid=1290151591, 2025. [Online; accessed 16-May-2025].

[29] SAE International. Sae levels of driving automation™ refined for clarity and international audience. https://www.sae.org/blog/sae-j3016-update, 2021. accessed 17-May-2025.

[30] Synopsys. What is an autonomous car? https://www.synopsys.com/glossary/what-is-autonomous-car.html. accessed 17-May-2025.

[31] Darsh Parekh, Nishi Poddar, Aakash Rajpurkar, Manisha Chahal, Neeraj Kumar, Gyanendra Prasad Joshi, and Woong Cho. A review on autonomous vehicles: Progress, methods and challenges. *Electronics*, 11(14):2162, 2022.

[32] Guoqiang Chen, Zhuangzhuang Mao, Huailong Yi, Xiaofeng Li, Bingxin Bai, Mengchao Liu, and Hongpeng Zhou. Pedestrian detection based on panoramic depth map transformed from 3d-lidar data. *Periodica Polytechnica Electrical Engineering and Computer Science*, 64(3):274–285, 2020.

[33] Walter. Autonomous vehicles: The technology behind the driverless revolution. https://apnews.ca/tech/autonomous-vehicles-the-technology-behind-the-driverless-revolution/#:~:text=Autonomous%20vehicles%2C%20also%20known%20as%20self-driving%20cars%2C%20rely,their%20environment%2C%20make%20decisions%2C%20and%20execute%20driving%20tasks., September 2024. [Online; accessed 17-May-2025].

[34] Dr Rajalakshmi Pachamuthu. Map-based navigation for autonomous vehicles. https://www.autonomousvehicleinternational.com/features/feature-map-based-navigation-for-autonomous-vehicles.html#:~:text=Map-based%20navigation%20is%20a%20critical%20component%20of%20autonomous,and%20localization%20technologies%20to%20operate%20safely%20and%20efficiently., September 2023. [Online; accessed 17-May-2025].

[35] Jui-An Yang and Chung-Hsien Kuo. Integrating vehicle positioning and path tracking practices for an autonomous vehicle prototype in campus environment. *Electronics*, 10(21):2703, 2021.

[36] Seong-Woo Kim, Gi-Poong Gwon, Woo-Sol Hur, Daejin Hyeon, Dae-Young Kim, Sung-Hyun Kim, Dong-Kyoung Kye, Sang-Hyun Lee, Soomok Lee, Myung-Ok Shin, et al. Autonomous campus mobility services using driverless taxi. *IEEE Transactions on intelligent transportation systems*, 18(12):3513–3526, 2017.

[37] Bernard Akindade Adaramola, Ayodeji Olalekan Salau, Favour Oluwatobi Adetunji, Olatomide Gbenga Fadodun, and Adebayo Tunbosun Ogundipe. Development and performance analysis of a gps-gsm guided system for vehicle tracking. In *2020 International Conference on Computation, Automation and Knowledge Management (ICCAKM)*, pages 286–290. IEEE, 2020.

[38] Ugwu Nnaemeka Virginus, Loveth Ijeoma Okafor, Ikechukwu Onyekachukwu, Jonathan Chukwunwike Agbo, Uzoigwe Charles Ikechukwu, Anyaorah Chukwuka Charles, Udechukwu Precious Emeka, Jonah Jane, and Obi Adaobi. Design and implementation vehicle tracking system using gsm and gps. *Int. J. Res. Innov. Appl. Sci*, 5:87–91, 2020.

[39] Shafri A Sharif, Mohd S Suhaimi, Nurul N Jamal, Irsyad K Riadz, Ikmal F Amran, and Dayang NA Jawawi. Real-time campus university bus tracking mobile application. In *2018 Seventh ICT International Student Project Conference (ICT-ISPC)*, pages 1–6. IEEE, 2018.

[40] Rezowana Akter, Md Jahid Hasan Khandaker, Shakil Ahmed, Muhtasim Munem Mugdho, and AKM Bahalul Haque. Rfid based smart transportation system with android application. In *2020 2nd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)*, pages 614–619. IEEE, 2020.

[41] M Hari Narasimhan, AL Reinhard Kenson, and S Vigneshwari. Smart bus management and tracking system. *Advances in Systems, Control and Automations: Select Proceedings of ETAEERE 2020*, 708:407, 2021.

[42] Thales Group. https://www.thalesgroup.com/sites/default/files/database/assets/images/2020-12/v2X.jpg, 2019. [Acessed on 5-22-2025].

[43] Bharat Bohara, Sunil Maharjan, and Bibek Raj Shrestha. Iot based smart home using blynk framework. *arXiv preprint arXiv:2007.13714*, 2020.

[44] NurAzira Jasman, Muhammad FarizzulIlham Mohammad Jalil, Azfarizal Mukhtar, KhairulSalleh Mohamed Sahari, and ME Rusli. Iot-based obstacle detection system for visually impaired person with smartphone module. *Journal of Advances in Information Technology*, 13(4), 2022.

[45] Dongliang You and Minjie Hu. A comparative study of cross-platform mobile application development. *Wellington, New Zealand*, 66, 2021.

[46] Wenhao Wu. React native vs flutter, cross-platforms mobile application frameworks. *Metropolia University of Applied Sciences*, 2018.

[47] Simon Stender and Hampus Åkesson. Cross-platform framework comparison: Flutter & react native, 2020.

[48] Deeplaxmi V Niture, Vivekanand Dhakane, Piyush Jawalkar, and Ankit Bamnote. Smart transportation system using iot. *Int. J. Eng. Adv. Technol*, 10(5):434–438, 2021.

[49] Sonam Khedkar, Swapnil Thube, WI Estate, C Naka, et al. Real time databases for applications. *International Research Journal of Engineering and Technology (IRJET)*, 4(06):2078–2082, 2017.

[50] GeeksForGeeks. What are software development methodologies — 15 key methodologies. https://www.geeksforgeeks.org/what-is-software-development-methodology-15-key-methodologies/, January 2024. [Acessed on 5-22-2025].

[51] Ken Schwaber and Jeff Sutherland. The scrum guide, 2020.

[52] Kanban University. What is kanban?, 2025.

[53] Extreme Programming. What is extreme programming?, 2025.

[54] Dimitar Dyankov. Software development techniques. https://www.thecodinghub.com/articles/software-development-techniques, April 2024. [Acessed on 5-22-2025].

[55] Barry W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988.

[56] James Martin. *Rapid Application Development*. Macmillan, 1991.

[57] Waymo. Waymo one: The first fully autonomous ride-hailing service. https://blog.waymo.com, 2023. Accessed: 2024-05-19.

[58] Cruise LLC. Cruise driverless rides now available in san francisco. https://getcruise.com, 2023. Accessed: 2024-05-19.

[59] Baidu. Apollo go: China's largest autonomous ride-hailing service. https://apollo.auto, 2024. Accessed: 2024-05-19.

[60] Motional and Lyft. Motional and lyft launch driverless rides in las vegas. https://motional.com/news, 2023. Accessed: 2024-05-19.

[61] Business+Tech. Startup spotlight: May mobility, 2020. Accessed: 2024-05-19.

[62] Beep Inc. Beep — starkville, ms autonomous shuttle, 2023. Accessed: 2024-05-19.

[63] University of Waterloo. University of waterloo launches canada's first driverless shuttle on campus, 2022. Accessed: 2024-05-19.

[64] KAUST Smart. Autonomous shuttle experience at kaust, 2021. Accessed: 2024-05-19.

[65] University of Western Australia. Uwa students first in australia to build brains of autonomous bus, 2021. Accessed: 2024-05-19.

[66] Wikipedia Contributors. Efeucampus – wikipedia, 2024. Accessed: 2024-05-19.